OPERATING SYSTEMS NOTES

Theory explained step by step

Author: SamuMGH04

WELCOME

Thanks for choosing these notes. This project is created with the aim of making people understand what's behind an Operating System and everything related to it.

LICENSE

These works are protected and distributed under the license
Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International

- **Attribution:** You must mention the original creator
- **Non Commercial:** You can't sell this material
- **No Derivatives:** You can't modify or change the text.

SISTEMI OPERATIVI: LEZIONE 1

Indice

1 Cosa è un sistema operativo? 12
1.1 Introduzione al concetto di Sistema Operativo . . . 12
1.2 Il Sistema Operativo come intermediario . . . 13
1.3 Obiettivo del Sistema Operativo . . . 13
1.3.1 Convenienza . . . 13
1.3.2 Efficienza . . . 14
1.4 Gestione delle Risorse Hardware . . . 14
1.5 Controllo dell'Esecuzione dei Programmi . . . 15
1.6 Prevenzione degli Errori e Uso Improprio . . . 15
1.7 Sintesi Concettuale . . . 16

2 Il sistema operativo all'interno del sistema informatico 16
2.1 Il Sistema Operativo come programma di controllo . . . 16
2.2 Gestione dell'operazione del sistema di calcolo . . . 17
2.3 Controllo dell'esecuzione dei programmi utente . . . 17
2.4 Gestione dei dispositivi hardware . . . 18
2.5 Coordinamento delle risorse fondamentali . . . 18
2.6 Correttezza ed efficienza del sistema . . . 19
2.7 Il flusso delle richieste dei programmi . . . 19
2.8 Conclusione della Sezione . . . 20

3 Obiettivi di un Sistema Operativo 20
3.1 Introduzione agli obiettivi del Sistema Operativo . . . 20
3.2 Esecuzione dei programmi utente e semplificazione della risoluzione dei problemi . . . 20
3.3 Fornire un ambiente di calcolo conveniente ed efficiente . . . 21
3.4 Uso efficiente delle risorse hardware . . . 21
3.5 Affidabilità, protezione e gestione corretta delle risorse . . . 21
3.6 Sintesi degli obiettivi . . . 22

4 Il Sistema Operativo come Gestore delle Risorse 22
4.1 Introduzione al ruolo di gestione delle risorse . . . 22
4.2 Prevenzione di errori e uso improprio del calcolatore . . . 22
4.3 Controllo dei dispositivi di input/output . . . 22
4.4 Garanzia di un comportamento ordinato ed efficiente del sistema . . . 23

5 I Molteplici Ruoli di un Sistema Operativo **23**
5.1 Introduzione ai ruoli del sistema operativo . . . 23
5.2 Diffusione dei computer e contesti d'uso . . . 23
5.3 Evoluzione dei sistemi operativi . . . 23
5.4 Sintesi . . . 24

6 Assenza di una Definizione Universale **24**
6.1 Introduzione . . . 24
6.2 Funzioni comuni dei sistemi operativi . . . 24
6.3 Il sistema operativo come software essenziale . . . 24
6.4 Osservazione finale . . . 25

7 Componenti di un Sistema Operativo **25**
7.1 Introduzione . . . 25
7.2 Il Kernel . . . 25
7.3 Programmi di sistema . . . 26
7.4 Programmi applicativi . . . 26
7.5 Middleware e framework moderni . . . 26
7.6 Sintesi . . . 26

8 Struttura di un Sistema di Calcolo **27**
8.1 Introduzione . . . 27
8.2 CPU e controller dei dispositivi . . . 27
8.3 Controller dei dispositivi e memoria locale . . . 27
8.4 Esecuzione parallela e concorrenza per la memoria . . . 27
8.5 Controllo della memoria condivisa . . . 28
8.6 Driver dei dispositivi . . . 28
8.7 Sintesi della struttura del sistema . . . 28

9 Evoluzione dei Microprocessori **29**
9.1 Introduzione . . . 29
9.2 Operazioni subnanosecondo . . . 29
9.3 Architettura multicore . . . 29
9.4 Cache multilevel . . . 29
9.5 Hyperthreading . . . 30
9.6 Computazione GPGPU . . . 30
9.7 Unità vettoriali . . . 30
9.8 DSP e SoC . . . 30
9.9 Sintesi . . . 31

10 Esecuzione delle Istruzioni **31**
10.1 Introduzione al ciclo di istruzione . . . 31
10.2 Fase di Fetch (Prelievo dell'Istruzione) . . . 31
10.3 Fase di Execution (Esecuzione dell'Istruzione) . . . 32
10.4 Tipologie di istruzioni . . . 32
10.5 Osservazioni . . . 32

11 Interruzioni (Interrupt) 33
11.1 Definizione di interruzione . . . 33
11.2 Uso delle interruzioni dai controller dei dispositivi . . . 33
11.3 Meccanismo di gestione di un'interruzione . . . 33
11.4 Vantaggi delle interruzioni . . . 33
11.5 Osservazioni finali . . . 34

12 Vettore di Interruzione (Interrupt Vector) 34
12.1 Introduzione . . . 34
12.2 Definizione di Interrupt Vector . . . 34
12.3 Funzionamento del vettore di interruzione . . . 34
12.4 Osservazioni . . . 35
12.5 Sintesi . . . 35

13 Ciclo di I/O Basato su Interruzioni (Interrupt-Driven I/O Cycle) 35
13.1 Introduzione . . . 35
13.2 Fasi del ciclo di I/O basato su interruzioni . . . 35
13.3 Vantaggi del ciclo di I/O basato su interruzioni . . . 36
13.4 Osservazioni finali . . . 36

14 Il Meccanismo Base delle Interruzioni 36
14.1 Introduzione . . . 36
14.2 Linea di richiesta di interruzione . . . 36
14.3 Identificazione e salto alla routine di servizio . . . 37
14.4 Funzioni della Interrupt Service Routine (ISR) . . . 37
14.5 Osservazioni . . . 37

15 Gestione Avanzata delle Interruzioni 38
15.1 Introduzione . . . 38
15.2 Ritardo della gestione delle interruzioni (Deferral) . . . 38
15.3 Dispatch efficiente alla ISR corretta . . . 38
15.4 Interruzioni a più livelli con priorità . . . 38
15.5 Implementazione hardware . . . 39
15.6 Osservazioni finali . . . 39

16 Gestione di Interruzioni Multiple 39
16.1 Introduzione . . . 39
16.2 Disabilitazione temporanea delle interruzioni . . . 39
16.3 Prioritizzazione delle interruzioni . . . 39
16.4 Sezioni critiche (Critical Regions) . . . 40
16.5 Comportamento tipico dei gestori di interruzioni . . . 40
16.6 Osservazioni finali . . . 40

17 Interruzioni Nidificate e Prioritarie 40
17.1 Introduzione . . . 40
17.2 Esempio: Processori ARM . . . 40
17.3 Funzionamento delle interruzioni nidificate . . . 41
17.4 Osservazioni . . . 41
17.5 Sintesi . . . 41

18 Interruzioni Mascherabili e Non Mascherabili **42**
18.1 Introduzione 42
18.2 Interruzioni non mascherabili (NMI) 42
18.3 Interruzioni mascherabili 42
18.4 Osservazioni finali 42

19 Catena di Interruzioni e Livelli di Priorità **43**
19.1 Introduzione 43
19.2 Catena di interruzioni (Interrupt Chaining) 43
19.3 Livelli di priorità delle interruzioni 43
19.4 Osservazioni 43

20 Tipi di Eccezioni **44**
20.1 Introduzione 44
20.2 Eccezioni rilevate dal processore (Processor-detected exceptions) 44
20.3 Eccezioni programmate (Programmed exceptions) 44
20.4 Osservazioni 44

21 Eccezioni Rilevate dal Processore: Fault, Trap e Abort **45**
21.1 Introduzione 45
21.2 Faults 45
21.3 Traps 45
21.4 Aborts 45
21.5 Osservazioni 45

22 Registri del Processore **46**
22.1 Introduzione 46
22.2 Registri visibili all'utente (User-Visible Registers) 46
22.3 Registri di controllo e stato (Control and Status Registers) 46
22.4 Osservazioni 46

23 Gerarchia della Memoria **47**
23.1 Introduzione 47
23.2 Principi fondamentali della gerarchia della memoria 47
23.3 Obiettivo della gerarchia della memoria 47
23.4 Osservazioni 47

24 Memoria Principale o RAM **48**
24.1 Introduzione 48
24.2 Implementazione 48
24.3 Accesso alla memoria 48
24.4 Volatilità della RAM 48
24.5 Osservazioni 48

25 Struttura Gerarchica della Memoria **49**
25.1 Introduzione 49
25.2 Trasferimento dei dati tra livelli di memoria 49
25.3 Caratteristiche dei livelli di memoria 49
25.4 Obiettivo della gerarchia 49

25.5 Osservazioni 49

26 Hit Ratio e Tempo Medio di Accesso 50
26.1 Introduzione 50
26.2 Hit Ratio 50
26.3 Tempi di accesso 50
26.4 Calcolo del tempo medio di accesso 50
26.5 Esempio pratico 50
26.6 Osservazioni 51

27 Località di Riferimento (Locality of Reference) 51
27.1 Introduzione 51
27.2 Caratteristiche principali 51
27.3 Dinamica della località 51
27.4 Osservazioni 51

28 Motivi delle Riferimenti Raggruppati (Reasons for Clustered References) 52
28.1 Introduzione 52
28.2 Cause principali dei riferimenti raggruppati 52
28.3 Osservazioni 52

29 Località Spaziale e Temporale (Spatial and Temporal Locality) 53
29.1 Introduzione 53
29.2 Località Spaziale (Spatial Locality) 53
29.3 Località Temporale (Temporal Locality) 53
29.4 Obiettivo della gestione della località 53
29.5 Osservazioni 53

30 Volatilità e Memoria Cache 54
30.1 Volatilità 54
30.2 Memoria Cache 54
30.3 Osservazioni 54

31 Memoria Virtuale e Cache del Disco 55
31.1 Memoria Virtuale 55
31.2 Cache del Disco 55
31.3 Osservazioni 55

32 Gestione della Memoria 56
32.1 Introduzione 56
32.2 Responsabilità principali del sistema operativo nella gestione della memoria 56
32.3 Memoria fisica e logica 57
32.4 Osservazioni 57

33 Larghezza di Banda della Memoria (Memory Bandwidth) 57
33.1 Introduzione 57
33.2 Problema della saturazione della larghezza di banda 57
33.3 Soluzione: NUMA 58
33.4 Osservazioni 58

34 Piattaforme Applicative (Application Platforms) 58
34.1 Introduzione 58
34.2 Esempio di piattaforma applicativa 58
34.3 Ruolo del Sistema Operativo 58
34.4 Chiamate di Sistema (System Calls) 59
34.5 Osservazioni 59

35 Ambienti Applicativi (Application Environments) 59
35.1 Sistemi Operativi Generali 59
35.2 Sistemi Embedded 60

36 Sistemi Real-Time (Real-Time Systems) 60
36.1 Introduzione 60
36.2 Classificazione dei RTOS 60
36.3 Osservazioni 60

37 Virtualizzazione (Virtualization) 61
37.1 Macchine Virtuali (Virtual Machines) 61
37.2 Ambienti Virtualizzati (Host Domains) 61
37.3 Osservazioni 61

38 Componenti Core di un Sistema Operativo (Core Components of an OS) 62
38.1 Evoluzione dei Sistemi Operativi 62
38.2 Componenti Core di un Sistema Operativo 62
38.3 Osservazioni 62

39 Componenti Tipici del Core di un Sistema Operativo (Typical Core Components of an Operating System) 63
39.1 Componenti Principali 63
39.2 Osservazioni 64

40 Importanza della Multiprogrammazione (Importance of Multiprogramming) 64
40.1 Introduzione 64
40.2 Concetti chiave 64
40.3 Scheduling dei Job 64
40.4 Osservazioni 65

41 Importanza del Multitasking (Importance of Multitasking) 65
41.1 Introduzione 65
41.2 Caratteristiche principali 65
41.3 Scheduling della CPU 65
41.4 Swapping della Memoria 65
41.5 Osservazioni 66

42 Gestione degli Errori nel Sistema Operativo: Modalità Doppia (Error Handling in the OS: Dual Mode) 66
42.1 Introduzione 66
42.2 Esempi di Errori ed Eccezioni 66

42.3 Modalità Doppia (Dual Mode) 66
42.4 Osservazioni 67

43 Gestione della Modalità Doppia (Dual-Mode Management) 67
43.1 Introduzione 67
43.2 Funzionamento del Bit di Modalità 67
43.3 Transizione tra Modalità 67
43.4 Osservazioni 67

44 Gestione dei Processi (Process Management) 68
44.1 Definizione di Processo 68
44.2 Risorse Necessarie a un Processo 68
44.3 Processi Single-Thread e Multi-Thread 68
44.4 Esecuzione Concurrente dei Processi 68
44.5 Responsabilità del Sistema Operativo nella Gestione dei Processi 69
44.6 Osservazioni 69

45 Gestione della Memoria Principale (Main Memory Management) 69
45.1 Introduzione 69
45.2 Responsabilità del Sistema Operativo nella Gestione della Memoria 70

46 Gestione della Memoria Secondaria (Secondary Memory Management) 70
46.1 Introduzione 70
46.2 Gestione dei Dispositivi di Memoria 70
46.3 Osservazioni 71

47 Gestione del File System (File System Management) 71
47.1 Introduzione 71
47.2 Responsabilità del Sistema Operativo nella Gestione dei File 71
47.3 Osservazioni 72

48 Gestione dei Dischi (Disk Management) 72
48.1 Introduzione 72
48.2 Responsabilità del Sistema Operativo nella Gestione dei Dischi 72
48.3 Osservazioni 72

49 Gestione della Memoria Terziaria (Tertiary Memory Management) 73
49.1 Introduzione 73
49.2 Caratteristiche dei Dispositivi di Memoria Terziaria 73
49.3 Osservazioni 73

50 Gestione dei Dispositivi di I/O (I/O Device Management) 74
50.1 Introduzione 74
50.2 Struttura del Sottosistema di I/O 74
50.3 Osservazioni 74

51 Protezione e Sicurezza (Protection and Security) 75
51.1 Protezione 75
51.2 Sicurezza 75
51.3 Identificazione e Controllo degli Accessi 75

51.4 Osservazioni 75

52 Ambienti Multiprogrammati (Multiprogrammed Environments) 76
52.1 Introduzione 76
52.2 Gestione dei Processi e dei Thread 76
52.3 Accesso ai Dispositivi di I/O 76
52.4 Osservazioni 76

53 Ambienti di Personal Computing (Personal Computing Environments) 77
53.1 Evoluzione degli Ambienti di Elaborazione 77
53.2 Office Automation 77
53.2.1 In passato 77
53.2.2 Oggi 77
53.3 Home Computing 77
53.3.1 In passato 77
53.3.2 Oggi 77
53.4 Osservazioni 78

54 Ambienti di Calcolo Distribuito (Distributed Computing Environments) 78
54.1 Definizione di Sistema Distribuito 78
54.2 Comunicazione 78
54.3 Vantaggi dei Sistemi Distribuiti 78
54.4 Osservazioni 79

55 Ambienti Distribuiti in Rete (Networked Distributed Environments) 79
55.1 Modello Client–Server 79
55.2 Servizi Offerti dai Server 79
55.3 Caratteristiche del Modello Client–Server 79
55.4 Osservazioni 79

56 Modello Peer-to-Peer (Peer-to-Peer Model) 80
56.1 Introduzione 80
56.2 Caratteristiche del Modello P2P 80
56.3 Esempi Famosi 80
56.4 Osservazioni 80

57 Obiettivi di un Sistema Operativo (Objectives of an Operating System) 81

58 Servizi del Sistema Operativo (Operating System Services) 81
58.1 Osservazioni 82

59 Servizi per l'Efficienza del Sistema (System Efficiency Services) 82
59.1 Allocazione delle Risorse (Resource Allocation) 82
59.2 Registrazione delle Attività (Logging) 83
59.3 Protezione e Sicurezza (Protection and Security) 83
59.4 Osservazioni 83

60 Shell e Esecuzione dei Comandi (Shells and Command Execution) 84
60.1 Introduzione . . . 84
60.2 Ruolo Principale (Main Role) . . . 84
60.3 Metodi di Esecuzione dei Comandi (Command Execution Methods) . . . 84
60.3.1 Comandi Integrati (Built-in Commands) . . . 84
60.3.2 Comandi Esterni (External Commands) . . . 84
60.4 Osservazioni . . . 85

61 Interfaccia Grafica Utente (Graphical User Interface) 85
61.1 Introduzione . . . 85
61.2 Caratteristiche Chiave (Key Features) . . . 85
61.3 Azioni dell'Utente (User Actions) . . . 85
61.4 Osservazioni . . . 85

62 Interazione Touchscreen nei Sistemi Mobili (Touchscreen Interaction in Mobile Systems) 86
62.1 Introduzione . . . 86
62.2 Idea Principale . . . 86
62.3 Gesti Comuni . . . 86
62.4 Osservazioni . . . 86

63 System Call 87
63.1 Introduzione . . . 87
63.2 Categorie di System Call (System Call Categories) . . . 87
63.2.1 Process Control . . . 87
63.2.2 File Management . . . 87
63.2.3 Device Management . . . 87
63.2.4 Information Management . . . 88
63.2.5 Communication . . . 88
63.2.6 Protection . . . 88
63.3 Osservazioni . . . 88
63.4 Esempio: Programma di Copia di File . . . 89
63.4.1 Sequenza Tipica di System Call . . . 89
63.4.2 Ciclo di Copia dei Dati . . . 89
63.4.3 Terminazione dell'Operazione . . . 89
63.4.4 Osservazione Chiave . . . 89

64 API e System Call (APIs and System Calls) 90
64.1 Perché i Programmatori Usano le API . . . 90
64.2 API Comuni . . . 90
64.3 Run-Time Environment (RTE) . . . 90
64.4 Osservazioni . . . 90

65 Controllo dei Processi (Process Control) 91
65.1 Operazioni Principali . . . 91
65.2 Terminazione dei Processi . . . 91
65.3 Gestione dei Crash . . . 91
65.4 Ritorno al Sistema . . . 91
65.5 Codice di Uscita (Exit Code) . . . 91

65.6 Osservazioni 92

66 System Call per la Gestione dei File (File Management System Calls) 92
66.1 Creazione ed Eliminazione dei File 92
66.2 Accesso ai File 92
66.3 Operazioni sui File 92
66.4 Gestione degli Attributi dei File 92
66.5 Idea Chiave 92

67 System Call per la Gestione dei Dispositivi (Device Management System Calls) 93
67.1 Controllo delle Risorse (Resource Control) 93
67.2 Operazioni di I/O 93
67.3 Unificazione tra File e Dispositivi 93
67.4 Idea Chiave 93

68 System Call per la Gestione delle Informazioni (Information Management System Calls) 94
68.1 Esempi di System Call di Informazioni Generali 94
68.2 Debug e Analisi 94
68.3 Informazioni sui Processi 94
68.4 Idea Chiave 94

69 System Call per la Comunicazione (Communication System Calls) 95
69.1 Message Passing 95
69.1.1 Identificazione 95
69.1.2 Gestione delle Connessioni 95
69.1.3 Comunicazione 95
69.2 Shared Memory 95
69.3 Idea Chiave 96

70 System Call per la Protezione (Protection System Calls) 96
70.1 Operazioni Tipiche 96
70.2 Obiettivo Principale 96
70.3 Osservazioni 96

71 Caricamento, Collegamento e Rilocazione (Loading, Linking, and Relocation) 97
71.1 Fasi del Processo di Caricamento 97
71.2 Esempio in UNIX 97
71.3 Formati Eseguibili 97
71.4 Idea Chiave 97

72 Struttura del Sistema Operativo: Design Monolitico (OS Structure: Monolithic Design) 98
72.1 Concetto di Modulo 98
72.2 Struttura Monolitica 98
72.3 Esempio: UNIX Originale 98
72.4 Idea Chiave 98

73 Linux e Kernel Monolitici (Linux and Monolithic Kernels) 99
73.1 Comunicazione tra Applicazioni e Kernel . . . 99
73.2 Kernel Monolitico . . . 99
73.3 Vantaggi del Kernel Monolitico . . . 99
73.4 Esempi di Sistemi con Kernel Monolitico . . . 99
73.5 Idea Chiave . . . 99

74 Design a Strati e Modulare del Sistema Operativo (Layered and Modular OS Design) 100
74.1 Sistemi Tightly vs Loosely Coupled . . . 100
74.2 Approccio a Strati (Layered Approach) . . . 100
74.3 Vantaggi del Design a Strati e Modulare . . . 100
74.4 Idea Chiave . . . 100

75 Architettura a Microkernel (Microkernel Architecture) 101
75.1 Origine ed Esempi . . . 101
75.2 Caratteristiche Chiave . . . 101
75.3 Vantaggi . . . 101
75.4 Idea Chiave . . . 101

76 Moduli Caricabili del Kernel (Loadable Kernel Modules) 102
76.1 Concetto Chiave . . . 102
76.2 Esempio di Moduli e Funzionalità . . . 102
76.3 Vantaggi . . . 102
76.4 Idea Chiave . . . 102

77 Sistemi Operativi Ibridi (Hybrid Operating Systems) 103
77.1 Concetto Chiave . . . 103
77.2 Esempi di Sistemi Ibridi . . . 103
77.3 Idea Chiave . . . 103

78 Generazione e Configurazione del Sistema Operativo (OS Generation & Configuration) 104
78.1 Fasi per Costruire un Sistema Operativo . . . 104
78.2 Metodi di Configurazione . . . 104
78.3 Idea Chiave . . . 104

1 Cosa è un sistema operativo?

1.1 Introduzione al concetto di Sistema Operativo

Un **sistema operativo** (*Operating System*, spesso abbreviato in **OS**) è un programma che svolge il ruolo di **intermediario** tra l'utente e l'hardware del calcolatore.

Per comprendere pienamente questa definizione è necessario chiarire il significato dei termini coinvolti:

- Per **utente** si intende qualunque entità che interagisce con il sistema di calcolo. L'utente può essere:
  - una persona fisica che utilizza una tastiera, un mouse o un'interfaccia grafica;
  - un altro programma applicativo che richiede servizi al sistema;
  - un processo automatico che opera senza intervento umano diretto.
- Per **hardware** si intende l'insieme delle componenti fisiche del calcolatore, quali:
  - la CPU (Central Processing Unit),
  - la memoria principale (RAM),
  - i dispositivi di memorizzazione secondaria (dischi, SSD),
  - i dispositivi di input/output (monitor, tastiera, stampante, rete, ecc.).

L'hardware, preso isolatamente, è costituito da circuiti elettronici capaci di eseguire istruzioni macchina, ma non possiede alcuna capacità autonoma di organizzare l'uso delle proprie risorse. È quindi necessario un software che:

1. coordini l'utilizzo delle risorse fisiche;
2. renda l'interazione con la macchina comprensibile e utilizzabile;
3. garantisca che l'esecuzione dei programmi avvenga in modo corretto e sicuro.

Il sistema operativo è esattamente questo software.

1.2 Il Sistema Operativo come intermediario

Affermare che il sistema operativo agisce come **intermediario** significa che esso si colloca logicamente tra due livelli:

$$\text{Utente} \longleftrightarrow \text{Sistema Operativo} \longleftrightarrow \text{Hardware}$$

L'utente non accede direttamente ai registri della CPU, né gestisce manualmente l'allocazione della memoria o il controllo dei dispositivi di I/O. Invece:

- L'utente interagisce con programmi applicativi (editor, browser, compilatori, ecc.).
- I programmi applicativi richiedono servizi al sistema operativo.
- Il sistema operativo traduce tali richieste in operazioni hardware concrete.

Questo modello garantisce:

- astrazione dell'hardware;
- semplicità d'uso;
- protezione delle risorse.

L'idea di *astrazione* è fondamentale: il sistema operativo fornisce una rappresentazione semplificata e controllata dell'hardware. Ad esempio, un file su disco viene presentato come un oggetto logico manipolabile, senza che l'utente debba conoscere la disposizione fisica dei settori magnetici.

1.3 Obiettivo del Sistema Operativo

Lo scopo principale del sistema operativo è fornire un ambiente che sia:

- **Conveniente** (*convenient*);
- **Efficiente** (*efficient*).

Analizziamo nel dettaglio questi due obiettivi.

1.3.1 Convenienza

Un ambiente è detto conveniente quando permette all'utente di eseguire programmi in modo semplice e intuitivo. Questo implica:

- interfacce chiare (grafiche o testuali);
- gestione automatica delle risorse;
- assenza della necessità di conoscere dettagli hardware complessi.

Senza un sistema operativo, l'utente dovrebbe:

- scrivere direttamente codice in linguaggio macchina;
- gestire manualmente memoria e periferiche;
- controllare esplicitamente ogni operazione di I/O.

Il sistema operativo elimina questa complessità, offrendo servizi standardizzati.

1.3.2 Efficienza

Un ambiente è efficiente quando utilizza le risorse hardware nel modo migliore possibile.

Le risorse principali sono:

- tempo di CPU,
- spazio di memoria,
- capacità di memorizzazione,
- dispositivi di input/output.

L'efficienza implica:

- riduzione dei tempi di attesa;
- utilizzo ottimale della CPU;
- minimizzazione degli sprechi di memoria;
- gestione concorrente di più programmi.

Il sistema operativo deve quindi bilanciare convenienza ed efficienza, due obiettivi che talvolta possono entrare in conflitto (ad esempio, maggiore protezione può comportare un leggero overhead computazionale).

1.4 Gestione delle Risorse Hardware

Una delle funzioni fondamentali del sistema operativo è la **gestione delle risorse hardware**.

Le risorse di un sistema sono limitate. Ad esempio:

- la memoria ha dimensione finita;
- la CPU può eseguire un numero limitato di istruzioni per unità di tempo;
- i dispositivi di I/O possono servire una richiesta alla volta.

Il sistema operativo agisce quindi come un **gestore di risorse** (resource manager), assegnando tali risorse ai programmi in esecuzione secondo criteri stabiliti.

Formalmente, possiamo interpretare il sistema operativo come una funzione di allocazione:

$$\text{OS} : \{\text{Richieste dei programmi}\} \longrightarrow \{\text{Assegnazioni di risorse}\}$$

Tale assegnazione deve rispettare:

- correttezza;
- equità;
- sicurezza;
- prestazioni.

1.5 Controllo dell'Esecuzione dei Programmi

Un'altra funzione centrale è il **controllo dell'esecuzione dei programmi**.

Quando un programma viene eseguito, può:

- tentare di accedere a memoria non autorizzata;
- entrare in un ciclo infinito;
- interferire con altri programmi;
- utilizzare impropriamente dispositivi hardware.

Il sistema operativo previene tali situazioni attraverso:

- meccanismi di protezione della memoria;
- gestione dei processi;
- controllo dei privilegi;
- isolamento tra programmi.

Questa funzione è essenziale per evitare:

- errori di sistema;
- comportamenti imprevedibili;
- uso improprio o dannoso del calcolatore.

1.6 Prevenzione degli Errori e Uso Improprio

Il sistema operativo non si limita a eseguire richieste, ma esercita un controllo attivo.

Quando un programma tenta un'operazione non consentita, il sistema operativo può:

- negare l'operazione;
- terminare il programma;
- generare un errore controllato;
- registrare l'evento per motivi di sicurezza.

Questo meccanismo di controllo garantisce:

- stabilità del sistema;
- integrità dei dati;
- protezione tra utenti diversi;
- affidabilità complessiva.

1.7 Sintesi Concettuale

Possiamo dunque riassumere formalmente il ruolo del sistema operativo come segue:

- È un **programma fondamentale** del sistema.
- Agisce come **intermediario** tra utente e hardware.
- Fornisce un ambiente **conveniente** ed **efficiente**.
- Gestisce e assegna le **risorse hardware**.
- Controlla l'esecuzione dei programmi.
- Previene errori e uso improprio del calcolatore.

Questa definizione costituisce il punto di partenza per lo studio sistematico dei sistemi operativi, i cui meccanismi interni verranno analizzati nei capitoli successivi.

2 Il sistema operativo all'interno del sistema informatico

2.1 Il Sistema Operativo come programma di controllo

Il sistema operativo è il **programma di controllo** che gestisce il funzionamento dell'intero sistema di calcolo.

Per comprendere pienamente questa affermazione è necessario chiarire cosa si intenda per *programma di controllo*. In generale, un programma di controllo è un software che:

- supervisiona l'esecuzione di altre attività;
- coordina componenti diverse;
- prende decisioni sull'uso delle risorse;
- garantisce il rispetto di regole e vincoli del sistema.

Nel contesto di un sistema di calcolo, il sistema operativo assume questa funzione globale: esso non svolge semplicemente un compito specifico, ma governa l'interazione tra tutte le componenti hardware e software.

2.2 Gestione dell'operazione del sistema di calcolo

Gestire l'operazione del sistema di calcolo significa assicurare che:

1. i programmi vengano caricati correttamente in memoria;
2. la CPU esegua le istruzioni nell'ordine appropriato;
3. le risorse siano allocate senza conflitti;
4. i dispositivi periferici funzionino in modo coordinato.

Senza un sistema operativo, ogni programma dovrebbe occuparsi autonomamente di:

- controllare direttamente le periferiche;
- gestire l'accesso alla memoria;
- sincronizzarsi con altri programmi.

Questo approccio sarebbe estremamente complesso e porterebbe facilmente a errori e conflitti. Il sistema operativo centralizza tali responsabilità, rendendo il sistema complessivamente stabile e coerente.

2.3 Controllo dell'esecuzione dei programmi utente

Il sistema operativo controlla l'esecuzione dei programmi utente. Questo implica diverse operazioni fondamentali:

- creazione del processo associato al programma;
- allocazione della memoria necessaria;
- assegnazione del tempo di CPU;
- gestione delle interruzioni e degli eventi;
- terminazione controllata del programma.

Il controllo dell'esecuzione non significa semplicemente "avviare" un programma, ma monitorarne costantemente il comportamento. Il sistema operativo deve assicurarsi che:

- il programma non acceda a memoria non autorizzata;
- non monopolizzi indefinitamente la CPU;
- non interferisca con altri programmi in esecuzione.

Questo controllo è reso possibile da meccanismi hardware e software, come modalità privilegiate di esecuzione e gestione delle interruzioni.

2.4 Gestione dei dispositivi hardware

Il sistema operativo gestisce i dispositivi hardware del sistema. I principali componenti coordinati sono:

- **CPU**: esegue le istruzioni dei programmi;
- **Memoria principale**: conserva temporaneamente dati e istruzioni;
- **Memoria secondaria**: permette la memorizzazione persistente;
- **Dispositivi di input/output (I/O)**: consentono l'interazione con l'esterno.

La gestione implica:

- allocazione delle risorse;
- prevenzione dei conflitti di accesso;
- ottimizzazione delle prestazioni;
- sincronizzazione tra operazioni concorrenti.

Ad esempio, se due programmi richiedono simultaneamente l'uso della stampante, il sistema operativo deve stabilire un ordine di esecuzione delle richieste, evitando sovrapposizioni o perdita di dati.

2.5 Coordinamento delle risorse fondamentali

Il sistema operativo coordina in modo integrato:

$$\text{CPU} \quad + \quad \text{Memoria} \quad + \quad \text{Storage} \quad + \quad \text{I/O}$$

Questo coordinamento è essenziale perché tali componenti non operano in modo isolato. Ogni programma in esecuzione richiede simultaneamente:

- tempo di CPU per eseguire istruzioni;
- spazio in memoria per dati e codice;
- accesso a file su memoria secondaria;
- comunicazione tramite dispositivi di I/O.

Il sistema operativo deve quindi garantire che l'interazione tra queste risorse avvenga:

- in modo corretto (assenza di errori);
- in modo coerente (assenza di conflitti logici);
- in modo efficiente (massimizzazione delle prestazioni).

2.6 Correttezza ed efficienza del sistema

Affermare che il sistema operativo assicura il funzionamento corretto ed efficiente del sistema significa che esso persegue simultaneamente due obiettivi fondamentali:

1. **Correttezza**: il sistema deve produrre risultati conformi alle specifiche e mantenere l'integrità dei dati.
2. **Efficienza**: le risorse devono essere utilizzate nel modo più produttivo possibile.

Questi due obiettivi non sono indipendenti. Una gestione scorretta delle risorse può compromettere l'efficienza; viceversa, una ricerca eccessiva di prestazioni può ridurre il livello di sicurezza o affidabilità.

Il sistema operativo realizza un compromesso progettuale tra tali esigenze.

2.7 Il flusso delle richieste dei programmi

Ogni richiesta proveniente da un programma — ad esempio:

- lettura di dati da un file;
- scrittura su disco;
- stampa di un documento;
- allocazione di memoria;
- comunicazione tramite rete;

deve necessariamente passare attraverso il sistema operativo.

Formalmente, possiamo rappresentare questo principio come:

$$\text{Programma Utente} \longrightarrow \text{Sistema Operativo} \longrightarrow \text{Hardware}$$

Il programma non interagisce direttamente con l'hardware, ma utilizza **servizi** forniti dal sistema operativo. Questa architettura garantisce:

- controllo centralizzato;
- sicurezza;
- astrazione;
- portabilità del software.

Se un programma vuole leggere dati da un disco, non accede fisicamente ai settori del supporto magnetico o a stato solido; esso invoca una funzione del sistema operativo, che traduce la richiesta in operazioni hardware concrete.

2.8 Conclusione della Sezione

All'interno del sistema di calcolo, il sistema operativo occupa una posizione centrale e strategica. Esso:

- governa l'intero funzionamento del sistema;
- controlla l'esecuzione dei programmi;
- gestisce e coordina tutte le risorse hardware;
- funge da unico canale di comunicazione tra software applicativo e componenti fisiche.

Questa centralità rende il sistema operativo l'elemento strutturale fondamentale dell'architettura di un sistema di calcolo moderno, costituendo la base su cui si sviluppano tutti gli altri livelli software.

3 Obiettivi di un Sistema Operativo

3.1 Introduzione agli obiettivi del Sistema Operativo

Il sistema operativo non esiste solo per coordinare l'hardware, ma ha obiettivi chiari e ben definiti che guidano la sua progettazione e implementazione. Comprendere questi obiettivi è fondamentale per analizzare il comportamento e le funzioni di un OS.

Gli obiettivi principali possono essere sintetizzati in quattro punti, ciascuno dei quali verrà analizzato dettagliatamente.

3.2 Esecuzione dei programmi utente e semplificazione della risoluzione dei problemi

Il primo obiettivo è permettere l'**esecuzione dei programmi utente** e rendere più semplice la **risoluzione dei problemi**.

- **Esecuzione dei programmi utente:** il sistema operativo deve fornire tutti i servizi necessari affinché un programma possa essere eseguito correttamente senza che l'utente debba gestire direttamente dettagli hardware.
- **Semplificazione della risoluzione dei problemi:** grazie ai servizi offerti, l'utente può concentrarsi sulla logica del problema da risolvere, senza preoccuparsi di allocazioni di memoria, gestione di file o sincronizzazione di dispositivi.

In altre parole, il sistema operativo agisce come uno strato di astrazione, traducendo le richieste dell'utente in operazioni hardware concrete.

3.3 Fornire un ambiente di calcolo conveniente ed efficiente

Il secondo obiettivo consiste nel fornire un **ambiente di calcolo conveniente ed efficiente**. Questo significa:

- **Convenienza:** l'utente può eseguire programmi e accedere alle risorse del sistema in modo semplice e intuitivo.
- **Efficienza:** il sistema utilizza le risorse disponibili (CPU, memoria, dispositivi di I/O) in modo ottimale, riducendo tempi morti e sprechi.

L'ambiente conveniente ed efficiente rappresenta il compromesso tra facilità d'uso e prestazioni ottimali, caratteristica essenziale per qualsiasi sistema operativo moderno.

3.4 Uso efficiente delle risorse hardware

Il terzo obiettivo è l'**uso efficiente delle risorse hardware**.

Il sistema operativo deve garantire:

- assegnazione ottimale della CPU tra processi concorrenti;
- gestione efficace della memoria principale e secondaria;
- coordinamento dei dispositivi di I/O per ridurre conflitti e attese;
- minimizzazione dello spreco di risorse limitate.

Il principio fondamentale è che le risorse sono finite e devono essere condivise tra più programmi e utenti senza compromettere l'integrità e le prestazioni del sistema.

3.5 Affidabilità, protezione e gestione corretta delle risorse

Il quarto obiettivo riguarda **affidabilità**, **protezione** e **gestione corretta delle risorse**.

- **Affidabilità:** il sistema deve funzionare correttamente anche in presenza di errori o condizioni impreviste, evitando crash e perdita di dati.
- **Protezione:** l'OS deve impedire che un programma o un utente interferisca con i dati o le risorse di altri, garantendo isolamento e sicurezza.
- **Gestione corretta delle risorse:** il sistema operativo deve monitorare l'uso delle risorse e prevenire conflitti, deadlock o uso improprio.

Questi tre aspetti assicurano che il sistema rimanga stabile, sicuro e coerente anche in scenari complessi con più utenti e programmi simultanei.

3.6 Sintesi degli obiettivi

In sintesi, possiamo riassumere gli obiettivi principali di un sistema operativo con il seguente elenco:

1. Eseguire i programmi utente e semplificare la risoluzione dei problemi.
2. Fornire un ambiente di calcolo conveniente ed efficiente.
3. Usare le risorse hardware in modo efficiente.
4. Garantire affidabilità, protezione e gestione corretta delle risorse.

Questi obiettivi costituiscono la guida concettuale che informa tutte le funzionalità, le strutture e i meccanismi implementati in un sistema operativo moderno.

4 Il Sistema Operativo come Gestore delle Risorse

4.1 Introduzione al ruolo di gestione delle risorse

Il sistema operativo è un **programma di controllo** che gestisce l'esecuzione dei programmi utente. La funzione di gestione delle risorse consiste nel coordinare in modo centralizzato tutte le componenti hardware e software, al fine di garantire che il sistema operi correttamente e in maniera efficiente.

4.2 Prevenzione di errori e uso improprio del calcolatore

Una delle responsabilità principali del sistema operativo è **prevenire errori e uso improprio del calcolatore**. In pratica:

- impedisce ai programmi di accedere a risorse non autorizzate;
- evita che un programma monopolizzi la CPU o la memoria;
- controlla che l'accesso ai dispositivi sia corretto e sicuro.

Questa funzione protegge sia l'integrità del sistema sia i dati degli utenti.

4.3 Controllo dei dispositivi di input/output

Il sistema operativo **controlla l'operazione dei dispositivi di input e output (I/O)**. Ciò significa che:

- tutte le richieste di lettura e scrittura passano attraverso il sistema operativo;
- il coordinamento delle periferiche evita conflitti e accessi simultanei non gestiti;
- le operazioni I/O sono organizzate in maniera ordinata, prevedibile e affidabile.

4.4 Garanzia di un comportamento ordinato ed efficiente del sistema

Il risultato finale della gestione delle risorse è che il sistema operativo **assicura un comportamento ordinato ed efficiente del sistema**.

- **Ordinato:** tutte le operazioni avvengono secondo regole precise, evitando interferenze tra programmi e dispositivi.
- **Efficiente:** le risorse hardware vengono utilizzate in modo ottimale, massimizzando le prestazioni complessive.

In sintesi, il sistema operativo come *resource manager* funge da supervisore centrale, garantendo stabilità, correttezza e prestazioni nel funzionamento dell'intero computer.

5 I Molteplici Ruoli di un Sistema Operativo

5.1 Introduzione ai ruoli del sistema operativo

Il termine **sistema operativo** copre una vasta gamma di **ruoli e funzioni**. Non si tratta di un singolo compito, ma di un insieme complesso di responsabilità che garantiscono il funzionamento coerente e affidabile del computer.

5.2 Diffusione dei computer e contesti d'uso

I computer sono strumenti polivalenti che servono a molteplici scopi e si trovano ormai ovunque, ad esempio:

- **Automobili:** sistemi di controllo, navigazione, gestione motore;
- **Veicoli spaziali:** controllo dei sistemi di bordo, gestione delle comunicazioni;
- **Case:** dispositivi smart, gestione energia, automazione domestica;
- **Aziende:** server, calcolo scientifico, gestione dati, applicazioni commerciali.

In ciascuno di questi contesti, il sistema operativo svolge un ruolo centrale nel rendere il computer utilizzabile e affidabile.

5.3 Evoluzione dei sistemi operativi

I sistemi operativi sono nati dalla necessità di rendere i computer **utilizzabili per compiti generali**, non limitati a una singola applicazione o scopo specifico.

Con l'avanzare della tecnologia, essi hanno subito un'evoluzione rapida, adattandosi a nuove architetture hardware, nuovi tipi di dispositivi e crescenti esigenze degli utenti.

5.4 Sintesi

Possiamo dunque affermare che un sistema operativo moderno:

- integra molteplici funzioni e ruoli;
- rende possibile l'uso general-purpose dei computer;
- si evolve continuamente per adattarsi a tecnologie e contesti d'uso sempre più diversificati.

Questa capacità di adattamento e molteplicità di ruoli è ciò che distingue i sistemi operativi moderni dai primi sistemi di calcolo, garantendo flessibilità, efficienza e affidabilità.

6 Assenza di una Definizione Universale

6.1 Introduzione

Non esiste una **definizione unica e universale** di sistema operativo. Questo perché i sistemi operativi coprono una varietà di ruoli, funzioni e contesti di utilizzo che variano in base al tipo di computer, all'hardware disponibile e alle esigenze dell'utente.

6.2 Funzioni comuni dei sistemi operativi

Nonostante l'assenza di una definizione unica, tutti i sistemi operativi condividono alcune funzioni fondamentali che rendono i computer utilizzabili:

1. **Controllo dei dispositivi di input/output:** garantire che dispositivi come tastiere, monitor, dischi e stampanti possano essere utilizzati dai programmi in modo sicuro e corretto.
2. **Gestione e allocazione delle risorse:** la CPU, la memoria e le periferiche devono essere condivise tra più programmi in esecuzione senza conflitti.
3. **Fornire un ambiente di esecuzione sicuro ed efficiente:** i programmi devono poter girare in modo ordinato, senza interferire tra loro e utilizzando le risorse nel modo più produttivo possibile.

6.3 Il sistema operativo come software essenziale

In sintesi, il sistema operativo è il **software essenziale** che trasforma l'hardware in uno strumento pratico e produttivo. Senza di esso, anche il computer più potente sarebbe praticamente inutilizzabile, perché richiederebbe all'utente di gestire manualmente tutte le risorse hardware e la loro interazione.

6.4 Osservazione finale

Il concetto di sistema operativo va quindi interpretato non come un insieme rigido di caratteristiche, ma come un **insieme di funzioni e servizi** progettati per:

- rendere l'hardware accessibile e funzionale;
- garantire la correttezza e la sicurezza dei programmi in esecuzione;
- massimizzare l'efficienza complessiva del sistema.

Questa definizione concettuale flessibile consente di includere tutte le varianti di sistemi operativi, dai dispositivi embedded ai grandi server e mainframe.

7 Componenti di un Sistema Operativo

7.1 Introduzione

Un sistema operativo non è un unico programma monolitico, ma è composto da diversi componenti che collaborano per fornire i servizi e le funzionalità necessarie. La comprensione di questi componenti è essenziale per analizzare l'architettura di un OS moderno.

7.2 Il Kernel

Il **kernel** rappresenta il **cuore** del sistema operativo. Caratteristiche principali:

- È il programma **core** che gira in modo continuo, anche quando altri programmi utente sono attivi o inattivi.
- Gestisce le **operazioni fondamentali** del computer, tra cui:
  - la gestione dei **processi**, assicurando che più programmi possano essere eseguiti contemporaneamente senza conflitti;
  - la gestione della **memoria**, allocando e proteggendo lo spazio necessario per ciascun programma;
  - la gestione dei **dispositivi hardware**, controllando I/O, dischi, periferiche e comunicazioni.

Il kernel opera a un livello privilegiato (modalità *kernel mode*), garantendo il pieno controllo sull'hardware e la sicurezza del sistema.

7.3 Programmi di sistema

I **programmi di sistema** sono utilities e strumenti che supportano il funzionamento complessivo del computer. Tra questi troviamo:

- strumenti per la gestione di file e directory;
- compilatori e interpreti;
- strumenti di diagnostica e monitoraggio del sistema;
- utility per backup, copia, e gestione delle risorse.

Questi programmi non sono parte del kernel, ma forniscono servizi essenziali per facilitare l'uso del sistema operativo e la manutenzione del sistema.

7.4 Programmi applicativi

I **programmi applicativi** sono software utilizzati per svolgere compiti specifici dell'utente. Esempi:

- editor di testo e fogli di calcolo;
- browser e client di posta elettronica;
- applicazioni di grafica, calcolo scientifico, gestione aziendale.

Il sistema operativo fornisce l'ambiente in cui questi programmi possono essere eseguiti correttamente e in sicurezza, offrendo astrazione dell'hardware e servizi comuni.

7.5 Middleware e framework moderni

Nei sistemi moderni, oltre ai tre componenti principali, possiamo trovare anche il **middleware**, cioè *software frameworks* che forniscono servizi aggiuntivi, tra cui:

- gestione grafica e interfacce utente avanzate;
- supporto multimediale (audio, video, streaming);
- gestione di basi di dati e servizi di rete.

Il middleware funge da strato intermedio tra sistema operativo e applicazioni, offrendo funzionalità avanzate senza richiedere modifiche dirette al kernel o ai programmi di base.

7.6 Sintesi

Possiamo quindi schematizzare l'architettura di un sistema operativo come segue:

Kernel → gestione core del sistema

Programmi di sistema → utilities e strumenti di supporto

Programmi applicativi → software per compiti utente

Middleware → servizi aggiuntivi avanzati (grafica, multimedia, database)

Questa struttura modulare consente al sistema operativo di essere flessibile, estensibile e capace di supportare una vasta gamma di applicazioni e hardware.

8 Struttura di un Sistema di Calcolo

8.1 Introduzione

Un sistema di calcolo moderno a uso generale è costituito da una combinazione di componenti hardware e software interconnessi. Comprendere la struttura fisica e logica di un computer è essenziale per analizzare il funzionamento del sistema operativo e la gestione delle risorse.

8.2 CPU e controller dei dispositivi

Un computer moderno è formato da:

- una o più **CPU** (Central Processing Unit), responsabili dell'esecuzione delle istruzioni;
- diversi **controller dei dispositivi**, ognuno dei quali gestisce un tipo specifico di periferica (ad esempio, disco, stampante, tastiera, scheda di rete);
- un **bus comune**, che permette la comunicazione tra CPU, controller e memoria condivisa.

Il **bus** è un canale di comunicazione condiviso che consente ai diversi componenti di accedere alla memoria e scambiare dati. La condivisione del bus richiede meccanismi di arbitraggio per prevenire conflitti tra CPU e controller.

8.3 Controller dei dispositivi e memoria locale

Ogni **controller dei dispositivi** ha caratteristiche specifiche:

- gestisce un particolare tipo di periferica;
- possiede **registri interni** per memorizzare comandi, stati e dati temporanei;
- contiene uno **storage locale** (buffer) per memorizzare temporaneamente i dati trasferiti tra CPU e dispositivo.

Questa architettura consente di ridurre il carico sulla CPU, poiché i controller possono gestire autonomamente alcune operazioni periferiche senza intervento diretto della CPU.

8.4 Esecuzione parallela e concorrenza per la memoria

La CPU e i controller dei dispositivi possono eseguire operazioni in parallelo, il che migliora le prestazioni complessive del sistema. Tuttavia, questo comporta una **concorrenza per l'accesso alla memoria condivisa**, poiché più componenti possono richiedere lettura o scrittura contemporaneamente.

8.5 Controllo della memoria condivisa

Per garantire accessi corretti e coordinati, è presente un **controller di memoria** che:

- sincronizza gli accessi alla memoria condivisa;
- previene conflitti e collisioni tra CPU e controller;
- ottimizza i cicli di memoria per massimizzare le prestazioni.

8.6 Driver dei dispositivi

Ogni periferica ha un **driver**, ovvero un programma software che fornisce al sistema operativo un'interfaccia uniforme per controllare il dispositivo.

Il driver consente:

- l'astrazione dell'hardware specifico, in modo che il sistema operativo possa interagire con dispositivi diversi tramite lo stesso insieme di comandi standard;
- la gestione di operazioni complesse (lettura, scrittura, interruzioni) senza che il sistema operativo debba conoscere i dettagli fisici del dispositivo.

8.7 Sintesi della struttura del sistema

In sintesi, la struttura di un computer moderno può essere schematizzata come segue:

$$\textbf{CPU} + \textbf{Controller dei dispositivi} \longleftrightarrow \textbf{Bus comune} \longleftrightarrow \textbf{Memoria condivisa}$$

Controller dei dispositivi $\rightarrow$ registri e buffer locali

Driver $\rightarrow$ interfaccia uniforme per il sistema operativo

Questa architettura permette l'esecuzione parallela, l'accesso efficiente alle risorse e la gestione coordinata di CPU, memoria e periferiche, garantendo il funzionamento ordinato ed efficiente del sistema.

9 Evoluzione dei Microprocessori

9.1 Introduzione

I microprocessori moderni hanno subito una notevole evoluzione, trasformandosi in sistemi di calcolo altamente paralleli ed estremamente efficienti. Questa evoluzione ha permesso di incrementare significativamente le prestazioni computazionali e di adattarsi a una vasta gamma di applicazioni, dai computer generici ai dispositivi embedded specializzati.

9.2 Operazioni subnanosecondo

I microprocessori contemporanei sono in grado di eseguire operazioni a **subnanosecondo**, cioè in tempi inferiori a un miliardesimo di secondo. Ciò consente:

- trasferimento dei dati a velocità eccezionali;
- incremento del **throughput** computazionale, ossia il numero di operazioni eseguite per unità di tempo;
- supporto a applicazioni ad alte prestazioni come calcolo scientifico, simulazioni e grafica.

9.3 Architettura multicore

I microprocessori moderni integrano più CPU, chiamate **core**, sullo stesso chip:

- ogni core può eseguire programmi in parallelo;
- l'architettura multicore permette l'esecuzione simultanea di più thread, aumentando l'efficienza e riducendo i colli di bottiglia della CPU;
- la progettazione multicore è fondamentale per il computing parallelo e per sfruttare al meglio le architetture software multi-threaded.

9.4 Cache multilevel

Per ridurre la latenza della memoria, i microprocessori includono **più livelli di cache**:

- cache **L1, L2, L3** integrate sul chip;
- memorie veloci che conservano copie temporanee dei dati più frequentemente utilizzati;
- diminuzione dei tempi di accesso rispetto alla memoria principale;
- aumento delle prestazioni complessive della CPU.

9.5 Hyperthreading

La tecnologia **hyperthreading** consente a ciascun core fisico di supportare più **processori logici**:

- le unità di esecuzione del core vengono condivise tra thread logici;
- questo aumenta l'utilizzo delle risorse interne e la parallelizzazione dei compiti;
- migliora le prestazioni per applicazioni multi-threaded senza aumentare il numero di core fisici.

9.6 Computazione GPGPU

I **GPU** (Graphics Processing Units) vengono utilizzati per **General-Purpose GPU computing (GPGPU)**:

- le GPU eseguono calcoli numerici utilizzando tecniche SIMD (Single Instruction, Multiple Data);
- sono particolarmente adatte a operazioni massivamente parallele, come simulazioni scientifiche e rendering grafico;
- integrano migliaia di unità di elaborazione in grado di operare simultaneamente su grandi dataset.

9.7 Unità vettoriali

Le CPU moderne integrano estensioni SIMD (Single Instruction, Multiple Data) per operazioni su array e matrici:

- consentono l'elaborazione parallela di vettori di dati;
- accelerano calcoli scientifici, elaborazioni multimediali e algoritmi di machine learning;
- sfruttano pipeline dedicate per aumentare il throughput.

9.8 DSP e SoC

Infine, l'evoluzione include processori specializzati come:

- **DSP (Digital Signal Processors)**: ottimizzati per flussi audio e video, elaborazioni di segnali e filtraggi digitali;
- **SoC (System-on-Chip)**: sistemi completi integrati su un singolo chip, utilizzati in dispositivi embedded come smartphone, automobili e dispositivi IoT.

9.9 Sintesi

In sintesi, l'evoluzione dei microprocessori ha portato a:

- operazioni estremamente veloci (*subnanosecondo*);
- architetture multicore e parallelismo avanzato;
- memoria cache multilivello per ridurre latenza;
- tecnologie hyperthreading per aumentare l'uso dei core;
- supporto a calcolo vettoriale e GPGPU;
- processori specializzati (DSP e SoC) per applicazioni dedicate.

Queste innovazioni rendono i microprocessori moderni strumenti estremamente potenti, capaci di soddisfare le esigenze sia dei computer generali sia dei sistemi embedded ad alte prestazioni.

10 Esecuzione delle Istruzioni

10.1 Introduzione al ciclo di istruzione

L'esecuzione delle istruzioni in un processore segue un **ciclo ripetitivo di istruzione** (*instruction cycle*), composto da due fasi principali: **fetch** ed **execution**. Questo ciclo rappresenta il comportamento fondamentale di qualsiasi CPU e descrive come il processore legge, interpreta ed esegue le istruzioni.

10.2 Fase di Fetch (Prelievo dell'Istruzione)

La prima fase del ciclo è la fase di **fetch**:

- Il processore **preleva la prossima istruzione dalla memoria**. Questo passaggio è necessario perché le istruzioni risiedono in memoria principale (RAM) e devono essere trasferite al processore per l'esecuzione.
- Il **Program Counter (PC)** contiene l'indirizzo della prossima istruzione da eseguire. Formalmente, se PC $= n$, il processore preleva l'istruzione situata all'indirizzo n della memoria.
- L'istruzione viene quindi caricata nel **Instruction Register (IR)**, dove resterà fino al completamento della fase di esecuzione.

Questa separazione tra prelievo e esecuzione consente alla CPU di gestire le istruzioni in modo ordinato e sequenziale, riducendo errori e conflitti.

10.3 Fase di Execution (Esecuzione dell'Istruzione)

La seconda fase è la fase di **execution**:

- Il processore **interpreta e esegue l'istruzione** caricata nell'IR.
- L'esecuzione continua finché il processore non subisce un **halt**, un errore o non viene spento.
- Durante l'esecuzione, la CPU può operare su registri, memoria o periferiche, a seconda del tipo di istruzione.

10.4 Tipologie di istruzioni

Le istruzioni possono essere classificate in quattro categorie principali, in base alla funzione che svolgono:

1. **Processor–Memory (CPU–Memoria):** trasferimento di dati tra processore e memoria principale. Esempi: `LOAD`, `STORE`.
2. **Processor–I/O (CPU–Dispositivi):** trasferimento di dati tra processore e periferiche di input/output. Esempi: lettura da tastiera, scrittura su stampante.
3. **Data Processing (Elaborazione dei dati):** operazioni aritmetiche (somma, sottrazione) o logiche (AND, OR, NOT) sui dati. Esempi: `ADD`, `SUB`, `AND`.
4. **Control (Controllo):** modifiche alla sequenza di esecuzione delle istruzioni. Esempi: salti condizionati, chiamate a subroutine, interrupt.

10.5 Osservazioni

- Il ciclo di istruzione è **ripetitivo**, in quanto il processore continua a prelevare ed eseguire istruzioni fino al termine del programma o allo spegnimento.
- La distinzione tra fetch ed execution è fondamentale per comprendere pipeline, parallelismo interno e ottimizzazioni moderne della CPU.
- Ogni istruzione è deterministica: dato lo stato della CPU, la memoria e i registri, l'esecuzione produrrà un risultato prevedibile.

11 Interruzioni (Interrupt)

11.1 Definizione di interruzione

Un'**interruzione** (*interrupt*) è un segnale inviato alla CPU per indicare che un evento richiede **attenzione immediata**. Questo meccanismo permette al processore di reagire prontamente a eventi esterni o interni senza doverli monitorare costantemente tramite polling.

11.2 Uso delle interruzioni dai controller dei dispositivi

I **controller dei dispositivi** utilizzano le interruzioni per comunicare con la CPU. Ad esempio:

- Informano la CPU che un'operazione di input/output è stata completata.
- Segnalano che si è verificato un errore nel dispositivo.

Grazie alle interruzioni, la CPU e i dispositivi possono operare in **concorrenza**, aumentando l'efficienza del sistema: la CPU non resta inattiva in attesa del completamento delle operazioni I/O, ma può continuare a eseguire altre istruzioni.

11.3 Meccanismo di gestione di un'interruzione

Quando si verifica un'interruzione, il sistema operativo e la CPU eseguono una sequenza ben definita:

1. La CPU **interrompe l'esecuzione del programma corrente** e salva il suo stato (registri, Program Counter, flag).
2. Il controllo viene trasferito alla **Interrupt Service Routine (ISR)** nel sistema operativo, un programma speciale progettato per gestire le interruzioni.
3. Il sistema operativo **determina la fonte dell'interruzione** e svolge il servizio richiesto, come completare un trasferimento dati o gestire un errore.
4. Al termine della gestione dell'interruzione, la CPU **ripristina lo stato salvato** e riprende l'esecuzione del programma utente interrotto.

11.4 Vantaggi delle interruzioni

Le interruzioni permettono di ottenere un **overlap efficiente** tra le operazioni di I/O e l'attività della CPU:

- La CPU non rimane inattiva in attesa che le operazioni di I/O vengano completate.
- I dispositivi possono operare in parallelo con l'esecuzione di programmi utente.
- Si ottimizzano le prestazioni complessive del sistema, riducendo i tempi morti e migliorando l'efficienza.

11.5 Osservazioni finali

- Il meccanismo di interruzione è alla base di sistemi multitasking e real-time, consentendo una risposta rapida agli eventi esterni.
- L'uso corretto delle ISR garantisce che gli eventi vengano gestiti in modo ordinato, evitando conflitti e perdite di dati.
- Le interruzioni rappresentano un esempio chiaro di come il sistema operativo coordini CPU e dispositivi, garantendo un funzionamento concorrente e stabile.

12 Vettore di Interruzione (Interrupt Vector)

12.1 Introduzione

Quando si verifica un'interruzione, la CPU deve sapere **quale routine eseguire** per gestire correttamente l'evento. Per fare ciò, utilizza una struttura chiamata **vettore di interruzione** (*interrupt vector*).

12.2 Definizione di Interrupt Vector

Un **vettore di interruzione** è una tabella in memoria che contiene gli **indirizzi di tutte le routine di servizio delle interruzioni** (ISR) presenti nel sistema.

Caratteristiche principali:

- Ogni **dispositivo o tipo di interruzione** ha un ingresso specifico nella tabella.
- Gli ingressi contengono l'indirizzo della **ISR** corrispondente.
- La tabella è generalmente riservata dal sistema operativo e protetta per evitare modifiche non autorizzate.

12.3 Funzionamento del vettore di interruzione

Quando un dispositivo genera un'interruzione:

1. La CPU riceve il segnale di interruzione insieme a un **numero identificativo dell'interruzione** (*interrupt number*).
2. La CPU utilizza questo numero per **accedere alla tabella del vettore di interruzione** e leggere l'indirizzo della ISR corrispondente.
3. Il controllo viene trasferito all'indirizzo specificato, permettendo al sistema operativo di gestire l'evento in modo corretto e ordinato.

12.4 Osservazioni

- Il vettore di interruzione rende possibile un **indirizzamento rapido e deterministico** delle routine di servizio.
- Grazie a questa struttura, la CPU non deve eseguire ricerche o controlli complessi per determinare quale routine avviare.
- Ogni dispositivo dispone del proprio ingresso nel vettore, garantendo che le interruzioni multiple possano essere gestite in modo separato e ordinato.

12.5 Sintesi

In sintesi, il **vettore di interruzione** è un elemento fondamentale per la gestione efficiente delle interruzioni, permettendo alla CPU di:

- identificare rapidamente l'ISR corretta;
- reagire prontamente agli eventi esterni;
- garantire il corretto funzionamento concorrente tra CPU e dispositivi.

13 Ciclo di I/O Basato su Interruzioni (Interrupt-Driven I/O Cycle)

13.1 Introduzione

Il **ciclo di I/O basato su interruzioni** è un meccanismo che consente alla CPU e ai dispositivi di input/output di operare in parallelo, migliorando l'efficienza complessiva del sistema. Questo approccio evita che la CPU debba attendere passivamente il completamento delle operazioni di I/O.

13.2 Fasi del ciclo di I/O basato su interruzioni

Il ciclo si sviluppa secondo i seguenti passaggi principali:

1. **Inizio dell'I/O da parte del driver del dispositivo:** Il *device driver* invia comandi al controller della periferica per iniziare il trasferimento dati.
2. **Trasferimento gestito dal controller:** Il *controller* della periferica esegue autonomamente il trasferimento dei dati, mentre la CPU continua a eseguire altre istruzioni senza attendere il completamento dell'operazione.
3. **Segnalazione di interruzione:** Una volta completata l'operazione di I/O o in caso di errore, il controller invia un **segnale di interruzione** alla CPU per informarla dell'avvenuto evento.
4. **Trasferimento del controllo all'ISR:** La CPU utilizza il **vettore di interruzione** per determinare l'indirizzo della *Interrupt Service Routine (ISR)* corrispondente e trasferisce il controllo a questa routine.

5. **Elaborazione dei dati da parte dell'ISR:** L'ISR gestisce i dati trasferiti, aggiorna eventuali registri o buffer e svolge le operazioni necessarie per completare il servizio richiesto.

6. **Ripresa del programma utente:** Al termine dell'esecuzione dell'ISR, la CPU **ripristina lo stato salvato** del programma interrotto e continua l'esecuzione del task originario.

13.3 Vantaggi del ciclo di I/O basato su interruzioni

- Consente alla CPU di lavorare su altri compiti mentre l'I/O è in corso, evitando tempi morti.
- Migliora l'**efficienza del sistema** e il throughput complessivo.
- Garantisce un **funzionamento ordinato e sicuro**, grazie all'uso delle ISR e del vettore di interruzione.
- Supporta la concorrenza tra CPU e dispositivi, rendendo possibile il multitasking e il calcolo parallelo.

13.4 Osservazioni finali

Il ciclo di I/O basato su interruzioni rappresenta una soluzione fondamentale nella progettazione dei sistemi operativi e delle architetture moderne, permettendo di bilanciare efficacemente il lavoro della CPU con le operazioni dei dispositivi e riducendo al minimo i colli di bottiglia nelle operazioni di input/output.

14 Il Meccanismo Base delle Interruzioni

14.1 Introduzione

Il **meccanismo base delle interruzioni** descrive come la CPU rileva e gestisce un'interruzione a livello hardware e software. Questo meccanismo costituisce il fondamento per l'esecuzione concorrente di CPU e dispositivi.

14.2 Linea di richiesta di interruzione

L'hardware della CPU include una **linea di richiesta di interruzione** (*interrupt-request line*):

- Dopo ogni istruzione, la CPU controlla questa linea per verificare se è stata attivata un'interruzione.
- L'attivazione avviene quando un controller di dispositivo necessita attenzione immediata.

Questo controllo costante consente alla CPU di rispondere rapidamente agli eventi esterni senza interrompere il ciclo di istruzione normale in modo inefficiente.

14.3 Identificazione e salto alla routine di servizio

Quando la linea di interruzione è attivata:

1. La CPU legge il **numero dell'interruzione** fornito dal controller.
2. Utilizzando il **vettore di interruzione**, la CPU individua l'indirizzo della *Interrupt Service Routine* (ISR) appropriata.
3. La CPU esegue un **salto** all'ISR per gestire l'evento.

14.4 Funzioni della Interrupt Service Routine (ISR)

La routine di servizio dell'interruzione esegue una sequenza precisa di operazioni:

1. **Salvataggio dello stato**: tutti i registri e lo stato della CPU vengono salvati per poter riprendere il programma interrotto senza perdita di informazioni.
2. **Determinazione della causa**: l'ISR identifica quale dispositivo o evento ha generato l'interruzione.
3. **Servizio del dispositivo**: l'ISR esegue le operazioni necessarie per completare la richiesta del dispositivo, come trasferire dati o segnalare errori.
4. **Ripristino dello stato**: lo stato della CPU salvato all'inizio viene ripristinato.
5. **Ritorno al programma utente**: la CPU riprende l'esecuzione del programma che era stato interrotto.

14.5 Osservazioni

- Il meccanismo garantisce che le interruzioni vengano gestite in modo ordinato e deterministico.
- La CPU può continuare a eseguire istruzioni con minima latenza di risposta agli eventi esterni.
- Questa struttura è alla base del multitasking e del funzionamento efficiente dei sistemi operativi moderni.

15 Gestione Avanzata delle Interruzioni

15.1 Introduzione

Nei sistemi moderni, le interruzioni non si limitano al meccanismo base; esistono funzionalità avanzate che permettono di gestire eventi in modo più efficiente e sicuro. Queste funzionalità sono integrate sia nell'hardware della CPU sia nei controller delle interruzioni.

15.2 Ritardo della gestione delle interruzioni (Deferral)

Una caratteristica fondamentale è la possibilità di **differire la gestione delle interruzioni** durante sezioni critiche del codice:

- Durante l'esecuzione di **critical sections**, cioè blocchi di codice che non devono essere interrotti, le interruzioni possono essere temporaneamente sospese.
- Ciò evita **inconsistenze o conflitti** nell'accesso a risorse condivise, come registri, memoria o dispositivi.
- Il sistema operativo riprende la gestione dell'interruzione non appena termina la sezione critica.

15.3 Dispatch efficiente alla ISR corretta

Nei sistemi moderni, la CPU e i controller delle interruzioni consentono un **dispatch rapido ed efficiente** verso la ISR appropriata:

- L'uso di vettori di interruzione ottimizzati permette alla CPU di saltare immediatamente all'indirizzo corretto.
- Riduce la latenza di risposta e minimizza il tempo speso per identificare quale routine eseguire.

15.4 Interruzioni a più livelli con priorità

Un'altra caratteristica avanzata è il supporto per **interruzioni multilivello con priorità**:

- Le interruzioni vengono classificate in base alla loro importanza.
- Una interruzione di livello superiore può sospendere temporaneamente una ISR di livello inferiore.
- Questo meccanismo garantisce che eventi critici ricevano attenzione immediata, migliorando l'affidabilità e la reattività del sistema.

15.5 Implementazione hardware

Queste funzionalità sono implementate tramite:

- **CPU moderne**, che includono registri di stato, maschere di interruzione e meccanismi per sospendere e riprendere interruzioni;
- **controller di interruzioni** dedicati, che gestiscono la priorità, il masking e l'instradamento degli eventi verso la CPU.

15.6 Osservazioni finali

- La gestione avanzata delle interruzioni è essenziale per sistemi multitasking e real-time.
- Migliora l'efficienza del sistema, garantendo che le risorse critiche non vengano compromesse.
- Permette di combinare sicurezza, reattività e parallelismo nell'esecuzione di CPU e dispositivi.

16 Gestione di Interruzioni Multiple

16.1 Introduzione

Nei sistemi moderni, possono verificarsi **più interruzioni contemporaneamente**. Il sistema operativo deve adottare strategie efficaci per decidere l'ordine e la modalità di gestione, garantendo efficienza e correttezza senza compromettere la reattività.

16.2 Disabilitazione temporanea delle interruzioni

Una strategia possibile è la **disabilitazione temporanea delle interruzioni**:

- Le nuove interruzioni vengono bloccate fino a quando quella corrente non è stata completamente gestita.
- Questo approccio è **semplice da implementare**, ma può essere rischioso in presenza di eventi critici per il tempo (*time-critical*).
- Può causare **perdita di dati** se i buffer I/O sono piccoli, poiché eventuali segnali aggiuntivi vengono ignorati durante la disabilitazione.

16.3 Prioritizzazione delle interruzioni

Un metodo più sofisticato consiste nella **prioritizzazione delle interruzioni**:

- Ogni interruzione riceve un **livello di priorità** stabilito dal sistema operativo o dall'hardware.
- Le interruzioni a priorità più alta possono **prevalere su quelle a priorità più bassa**, interrompendo la routine corrente se necessario.
- Questo meccanismo garantisce che eventi critici siano gestiti prima di quelli meno urgenti, migliorando la reattività complessiva del sistema.

16.4 Sezioni critiche (Critical Regions)

Le **critical regions** sono parti del codice del kernel in cui le interruzioni vengono disabilitate:

- Devono essere **minime e brevi** per mantenere elevata la reattività del sistema.
- Consentono di proteggere risorse condivise come registri, strutture dati o buffer di memoria senza introdurre inconsistenze.

16.5 Comportamento tipico dei gestori di interruzioni

I **gestori di interruzioni** (ISR) seguono una prassi comune:

- All'inizio, le interruzioni vengono generalmente **disabilitate** per evitare conflitti.
- Successivamente, vengono **riabilitate il prima possibile** per permettere la gestione di altre interruzioni, evitando ritardi e colli di bottiglia.

16.6 Osservazioni finali

- La gestione di interruzioni multiple richiede un bilanciamento tra sicurezza (evitare conflitti) e reattività (rispondere rapidamente agli eventi critici).
- L'uso combinato di disabilitazione temporanea, priorità e sezioni critiche ottimizza l'efficienza del sistema e garantisce la correttezza dei dati.
- Queste strategie sono alla base dei sistemi operativi multitasking e real-time moderni.

17 Interruzioni Nidificate e Prioritarie

17.1 Introduzione

Alcune architetture di processore supportano le **interruzioni nidificate** (*nested interrupts*), che consentono a eventi ad alta priorità di interrompere gestori di interruzioni a priorità più bassa. Questo aumenta la reattività del sistema, specialmente in applicazioni real-time.

17.2 Esempio: Processori ARM

Nei processori ARM, le interruzioni sono gestite con due livelli principali:

- **IRQ (Interrupt Request)**: interruzioni normali, di priorità standard.
- **FIQ (Fast Interrupt Request)**: interruzioni veloci e ad alta priorità.

Caratteristiche principali:

- Ogni livello dispone di **banca di registri separata** e di un **vettore di interruzione dedicato**, permettendo l'esecuzione rapida senza sovrascrivere lo stato di registri critici.
- **Gestore IRQ**: durante la sua esecuzione, le interruzioni IRQ sono disabilitate, mentre le FIQ rimangono abilitate, permettendo a un evento FIQ di interrompere il gestore IRQ.
- **Gestore FIQ**: durante l'esecuzione, sia IRQ che FIQ sono disabilitate per garantire l'esecuzione immediata e non interrotta dell'evento ad alta priorità.

17.3 Funzionamento delle interruzioni nidificate

- Quando una FIQ si verifica mentre un IRQ è in esecuzione, la CPU **preempta** il gestore IRQ e passa immediatamente all'esecuzione della FIQ.
- Questo meccanismo consente di dare priorità assoluta agli eventi critici, riducendo al minimo la latenza di risposta.

17.4 Osservazioni

- Le interruzioni nidificate aumentano la complessità del sistema, ma migliorano notevolmente la reattività.
- L'uso di registri separati per FIQ e IRQ evita conflitti e riduce il tempo necessario per salvare e ripristinare lo stato della CPU.
- Questo schema è particolarmente utile nei sistemi embedded e real-time, dove gli eventi ad alta priorità devono essere gestiti immediatamente.

17.5 Sintesi

In sintesi, le interruzioni nidificate e prioritarie permettono:

- gestione simultanea di più livelli di eventi;
- preemption di interruzioni a bassa priorità da parte di quelle ad alta priorità;
- riduzione della latenza e aumento della reattività del sistema.

18 Interruzioni Mascherabili e Non Mascherabili

18.1 Introduzione

La maggior parte delle CPU moderne dispone di due linee di richiesta di interruzione, che differenziano gli eventi in base alla loro criticità e alla possibilità di essere temporaneamente disabilitati:

- **Interruzione non mascherabile (Nonmaskable Interrupt, NMI)**: riservata ad eventi critici, come guasti hardware o situazioni di emergenza, che devono essere gestiti immediatamente.
- **Interruzione mascherabile**: può essere temporaneamente disabilitata durante l'esecuzione di sezioni critiche o codice del kernel, evitando conflitti o corruzione di dati.

18.2 Interruzioni non mascherabili (NMI)

- Sono **sempre attive** e non possono essere ignorate dalla CPU.
- Vengono utilizzate per eventi che richiedono attenzione immediata e assoluta, come errori hardware gravi, guasti di memoria o watchdog timer.
- Il sistema operativo deve prevedere routine specifiche per gestire tali interruzioni in modo affidabile.

18.3 Interruzioni mascherabili

- Possono essere **temporaneamente disabilitate** per proteggere risorse condivise durante sezioni critiche del kernel.
- Sono comunemente usate dai **controller dei dispositivi** per richiedere servizi, come il completamento di un'operazione di I/O.
- Una volta completata la sezione critica, le interruzioni mascherabili vengono riabilitate, permettendo al sistema operativo di gestire eventuali richieste pendenti.

18.4 Osservazioni finali

- La distinzione tra NMI e interruzioni mascherabili è fondamentale per garantire **sicurezza, reattività e affidabilità** del sistema.
- Le NMI assicurano che eventi critici vengano sempre gestiti, mentre le interruzioni mascherabili permettono flessibilità e controllo nella gestione di eventi meno urgenti.
- L'uso combinato di queste due tipologie di interruzione consente un equilibrio ottimale tra reattività immediata e protezione delle risorse critiche.

19 Catena di Interruzioni e Livelli di Priorità

19.1 Introduzione

Nei sistemi complessi, è possibile che più dispositivi condividano la stessa linea di interruzione o che sia necessario gestire numerosi eventi simultanei. Per affrontare questa situazione, si utilizzano due meccanismi principali: la **catena di interruzioni** (*interrupt chaining*) e i **livelli di priorità delle interruzioni**.

19.2 Catena di interruzioni (Interrupt Chaining)

- In questo approccio, ogni ingresso del **vettore di interruzione** punta a una **lista di gestori** (*handlers*) associati a diversi dispositivi.
- Quando si verifica un'interruzione, il sistema richiama ciascun gestore in sequenza fino a quando uno di essi **servizio la richiesta** correttamente.
- Questo metodo è utile quando più dispositivi condividono lo stesso numero di interruzione, permettendo di individuare rapidamente quale dispositivo richiede attenzione.

19.3 Livelli di priorità delle interruzioni

- Le interruzioni possono essere assegnate a **livelli di priorità**, così che un'interruzione ad alta priorità possa **prevalere** su una a priorità più bassa.
- Questo meccanismo garantisce che eventi critici ricevano immediata attenzione anche se altri gestori sono in esecuzione.
- Combinando priorità e catena di interruzioni, il sistema operativo può mantenere alta **scalabilità e reattività**.

19.4 Osservazioni

- La combinazione di **interrupt chaining** e **priorità** consente di gestire efficientemente numerosi dispositivi su linee condivise.
- Assicura che le interruzioni vengano servite nell'ordine corretto e che le operazioni critiche siano trattate tempestivamente.
- Questi meccanismi sono essenziali nei sistemi multitasking e real-time, dove la tempestività e la gestione ordinata delle interruzioni sono fondamentali.

20 Tipi di Eccezioni

20.1 Introduzione

Le **eccezioni** sono eventi che modificano il normale flusso di esecuzione delle istruzioni. A differenza delle interruzioni, che sono solitamente generate da dispositivi esterni, le eccezioni possono originarsi sia dalla CPU sia dal software. Comprendere le eccezioni è essenziale per la gestione degli errori, il debugging e le chiamate di sistema.

20.2 Eccezioni rilevate dal processore (Processor-detected exceptions)

- Queste eccezioni sono **generate automaticamente dalla CPU** quando si verifica una condizione anomala durante l'esecuzione di un'istruzione.
- Tipologie principali:
  - **Faults**: errori che possono essere recuperati e che permettono il ripristino e la ripetizione dell'istruzione.
  - **Traps**: eventi rilevati volontariamente al termine dell'esecuzione dell'istruzione; spesso usati per il debugging.
  - **Aborts**: errori gravi che rendono impossibile continuare l'esecuzione dell'istruzione; tipicamente causati da malfunzionamenti hardware o violazioni di protezione.

20.3 Eccezioni programmate (Programmed exceptions)

- Queste eccezioni si verificano **su richiesta del programmatore**.
- Sono attivate intenzionalmente tramite **istruzioni speciali**, spesso chiamate *software interrupts*.
- Applicazioni principali:
  - **System calls**: per richiedere servizi al sistema operativo.
  - **Debugging**: per monitorare o controllare l'esecuzione di programmi.

20.4 Osservazioni

- Le eccezioni garantiscono che eventi anomali o richieste software possano essere gestiti in modo strutturato e sicuro.
- A differenza delle interruzioni, alcune eccezioni possono essere previste o programmate, consentendo al programmatore di controllare il flusso del programma.
- La distinzione tra eccezioni rilevate dal processore e programmate è fondamentale per progettare sistemi operativi affidabili e per la gestione dei casi di errore e debugging.

21 Eccezioni Rilevate dal Processore: Fault, Trap e Abort

21.1 Introduzione

Le eccezioni rilevate dal processore (*processor-detected exceptions*) rappresentano condizioni anomale individuate dalla CPU durante l'esecuzione delle istruzioni. Queste eccezioni sono classificate in tre categorie principali: **Faults**, **Traps** e **Aborts**. Ogni categoria ha caratteristiche specifiche e modalità di gestione distinte.

21.2 Faults

- I **faults** sono condizioni anomale che **possono essere corrette** dal sistema operativo o dall'hardware.
- Dopo la correzione, l'istruzione che ha generato il fault viene **riavviata** senza perdita di continuità, come se non si fosse mai verificato l'errore.
- Esempi tipici includono page faults, dove una pagina di memoria deve essere caricata dalla memoria secondaria.

21.3 Traps

- I **traps** si verificano quando **non è necessario rieseguire** l'istruzione che li ha generati.
- Sono spesso utilizzati per scopi di **debugging**, ad esempio i breakpoint.
- L'esecuzione del programma riprende con **l'istruzione successiva**, garantendo il monitoraggio o la gestione di eventi senza interrompere il flusso principale.

21.4 Aborts

- Gli **aborts** indicano errori gravi, come guasti hardware o tabelle di sistema inconsistenti.
- Queste condizioni sono **irrecuperabili** per l'istruzione corrente.
- Il gestore dell'abort solitamente **termina il processo interessato**, proteggendo il resto del sistema da possibili effetti collaterali.

21.5 Osservazioni

- La distinzione tra Faults, Traps e Aborts permette al sistema operativo di reagire in modo appropriato a diverse anomalie.
- Faults consentono il recupero e la continuità, Traps servono per monitoraggio e controllo, mentre Aborts proteggono il sistema da condizioni critiche.
- La gestione corretta di queste eccezioni è essenziale per garantire l'affidabilità e la sicurezza del sistema.

22 Registri del Processore

22.1 Introduzione

La CPU contiene diversi **registri**, che offrono una memoria temporanea ad accesso estremamente veloce. I registri sono essenziali per l'esecuzione efficiente delle istruzioni, poiché permettono di memorizzare dati e informazioni di controllo senza ricorrere alla memoria principale, molto più lenta.

22.2 Registri visibili all'utente (User-Visible Registers)

- Questi registri sono **utilizzati dai programmi** come locazioni di memoria temporanea.
- Permettono al software di memorizzare dati intermedi, risultati di operazioni aritmetiche o logiche, e puntatori temporanei.
- Sono **direttamente accessibili alla velocità della CPU**, offrendo prestazioni molto superiori rispetto all'uso della RAM.
- Esempi includono registri generali, contatori di ciclo o registri per indici e puntatori.

22.3 Registri di controllo e stato (Control and Status Registers)

- Questi registri sono **utilizzati dal sistema operativo** per controllare il funzionamento della CPU.
- Contengono informazioni fondamentali come:
  - **Program Counter (PC)**: indirizzo della prossima istruzione da eseguire.
  - **Instruction Register (IR)**: istruzione corrente in fase di esecuzione.
  - **Status Flags**: indicano condizioni come zero, overflow, carry o stato di interruzione.
- L'accesso a questi registri è tipicamente **privilegiato**, ossia consentito solo al sistema operativo o a istruzioni in modalità kernel, per evitare modifiche non autorizzate e garantire la sicurezza e stabilità del sistema.

22.4 Osservazioni

- La distinzione tra registri visibili all'utente e registri di controllo/stato permette al sistema operativo di mantenere il controllo sul processore senza compromettere le prestazioni dei programmi utente.
- L'uso efficiente dei registri è cruciale per l'ottimizzazione delle prestazioni della CPU e per la gestione delle istruzioni.
- Questi registri sono la base per operazioni come il salvataggio dello stato della CPU durante interruzioni, eccezioni o cambio di contesto.

23 Gerarchia della Memoria

23.1 Introduzione

La **gerarchia della memoria** è una struttura organizzativa che classifica i diversi tipi di memoria in base a tre parametri principali: **capacità**, **velocità** e **costo per bit**. La progettazione della memoria mira a fornire un equilibrio ottimale tra questi fattori per ottenere prestazioni elevate del sistema.

23.2 Principi fondamentali della gerarchia della memoria

- **Capacità crescente:** all'aumentare della capacità di memoria disponibile, le applicazioni possono espandere il loro utilizzo di dati e istruzioni, migliorando la produttività del sistema.
- **Velocità crescente:** le memorie più vicine alla CPU devono avere velocità più elevate per sostenere il ritmo del processore e prevenire colli di bottiglia.
- **Costo per bit:** memorie di grande capacità hanno tipicamente costi più bassi per bit, ma operano a velocità inferiori rispetto alle memorie più piccole e rapide.

23.3 Obiettivo della gerarchia della memoria

L'obiettivo principale della gerarchia della memoria è trovare un **compromesso ottimale tra costo, velocità e capacità**, al fine di:

- garantire un accesso rapido ai dati più frequentemente utilizzati,
- minimizzare i tempi di attesa della CPU,
- mantenere il costo totale del sistema entro limiti ragionevoli.

23.4 Osservazioni

- La gerarchia tipica include registri, cache di diversi livelli, memoria principale (RAM), memoria secondaria (dischi) e archivi a lungo termine.
- L'organizzazione gerarchica sfrutta la località dei dati: dati e istruzioni più frequentemente usati sono memorizzati nelle memorie più veloci.
- Un design ben bilanciato della gerarchia della memoria è fondamentale per le prestazioni complessive del sistema e per evitare che la CPU resti inattiva in attesa di dati.

24 Memoria Principale o RAM

24.1 Introduzione

La **memoria principale**, comunemente nota come **RAM** (*Random Access Memory*), è la memoria utilizzata dalla CPU per archiviare temporaneamente dati e istruzioni durante l'esecuzione dei programmi. La RAM è un elemento essenziale della gerarchia della memoria, garantendo accessi rapidi e diretti alle informazioni necessarie al processore.

24.2 Implementazione

- La RAM è implementata utilizzando la tecnologia **DRAM** (*Dynamic Random Access Memory*), un tipo di semiconduttore che memorizza ogni bit in un condensatore e necessita di refresh periodico per mantenere i dati.
- La memoria principale fornisce un **array di byte**, ciascuno con un **indirizzo unico** che permette alla CPU di leggere o scrivere informazioni in locazioni specifiche.

24.3 Accesso alla memoria

- Durante l'esecuzione di un programma, il processore **legge e scrive dati** agli indirizzi di memoria corrispondenti.
- Questo accesso diretto alle locazioni di memoria consente operazioni veloci e deterministiche, fondamentali per le prestazioni della CPU.

24.4 Volatilità della RAM

- La memoria principale è **volatile**, il che significa che i dati vengono persi quando l'alimentazione elettrica viene interrotta.
- Per la conservazione permanente delle informazioni, sono necessari dispositivi di memoria secondaria come dischi, SSD o memorie di tipo non volatile.

24.5 Osservazioni

- La RAM è la memoria più veloce a disposizione della CPU per dati temporanei, pur essendo più costosa per bit rispetto alla memoria di massa.
- La sua organizzazione in byte con indirizzi unici facilita l'implementazione di strutture dati e l'accesso casuale, essenziale per programmi complessi e multitasking.
- La combinazione di RAM volatile e memoria secondaria non volatile costituisce la base della gerarchia della memoria nei sistemi moderni.

25 Struttura Gerarchica della Memoria

25.1 Introduzione

La **struttura gerarchica della memoria** sfrutta la differenza di velocità e costo tra diversi livelli di memoria per ottenere un accesso rapido ai dati più frequentemente utilizzati, pur mantenendo capacità elevate e costi contenuti. Questo concetto è alla base della progettazione moderna dei sistemi di memoria.

25.2 Trasferimento dei dati tra livelli di memoria

- I dati vengono trasferiti dalla memoria **più lenta** e di grande capacità (M_2) verso la memoria **più veloce** e vicina alla CPU (M_1).
- Questo trasferimento permette al processore di accedere rapidamente ai dati critici senza attendere i tempi più lunghi della memoria di livello inferiore.

25.3 Caratteristiche dei livelli di memoria

- **Memoria veloce (M_1)**:
  - Piccola capacità.
  - Costosa per bit.
  - Accesso estremamente rapido, vicina alla velocità della CPU.
- **Memoria lenta (M_2)**:
  - Grande capacità.
  - Più economica per bit.
  - Accesso più lento rispetto a M_1.

25.4 Obiettivo della gerarchia

L'obiettivo principale della struttura gerarchica è:

- **Ottenere la velocità della memoria** M_1 per i dati frequentemente utilizzati.
- **Conservare la capacità e l'efficienza dei costi di** M_2 per i dati meno critici o meno frequentemente accessibili.

25.5 Osservazioni

- La gerarchia memoria permette di combinare le migliori caratteristiche di due tipi di memoria: velocità e capacità.
- La gestione dei trasferimenti tra M_2 e M_1 può avvenire tramite meccanismi hardware (cache) o software (gestione dei buffer e paging).
- Un sistema ben progettato massimizza l'uso della memoria veloce, riducendo i tempi di attesa della CPU e migliorando il throughput complessivo.

26 Hit Ratio e Tempo Medio di Accesso

26.1 Introduzione

Nella gerarchia della memoria, non tutti i dati si trovano sempre nella memoria più veloce (M_1). La **hit ratio** (H) e il **tempo medio di accesso** (T_s) sono metriche fondamentali per valutare l'efficienza di un sistema di memoria gerarchica.

26.2 Hit Ratio

- La **hit ratio** H rappresenta la frazione di accessi alla memoria in cui i dati richiesti si trovano già nella memoria veloce M_1.
- Valori di H più elevati indicano una maggiore probabilità di trovare i dati in memoria veloce, riducendo la necessità di accedere alla memoria più lenta M_2.

26.3 Tempi di accesso

- T_1 = tempo di accesso alla memoria veloce M_1
- T_2 = tempo di accesso alla memoria lenta M_2

26.4 Calcolo del tempo medio di accesso

Il **tempo medio di accesso** T_s combina la probabilità di trovare i dati in M_1 con il tempo necessario in caso contrario:

$$T_s = H \times T_1 + (1 - H) \times (T_1 + T_2)$$

Semplificando l'equazione:

$$T_s = T_1 + (1 - H) \times T_2$$

26.5 Esempio pratico

Supponiamo i seguenti valori:

- $T_1 = 0.1\,\mu s$
- $T_2 = 1.0\,\mu s$
- $H = 0.95$

Allora il tempo medio di accesso è:

$$T_s = 0.95 \times 0.1 + 0.05 \times (0.1 + 1.0) = 0.15\,\mu s$$

26.6 Osservazioni

- Maggiore è H, più il tempo medio di accesso T_s si avvicina a T_1, il tempo della memoria più veloce.
- Questa relazione evidenzia l'importanza di massimizzare la hit ratio tramite cache efficaci o politiche di gestione della memoria.
- Il calcolo del tempo medio permette di prevedere le prestazioni complessive del sistema e di ottimizzare la gerarchia memoria.

27 Località di Riferimento (Locality of Reference)

27.1 Introduzione

La **località di riferimento** è un principio fondamentale nella progettazione dei sistemi di memoria e cache. Esso descrive il comportamento della CPU nell'accedere ripetutamente a un insieme limitato di locazioni di memoria per un certo intervallo di tempo.

27.2 Caratteristiche principali

- La maggior parte degli accessi alla memoria avviene in **locazioni vicine tra loro**.
- La CPU tende a lavorare con un **piccolo insieme di indirizzi** per un certo periodo, prima di passare ad altre zone.
- Questo modello di accesso si applica sia alle **istruzioni** sia ai **dati**.

27.3 Dinamica della località

- Nel tempo, le **aree attive** della memoria cambiano, seguendo il flusso del programma verso nuove sezioni di codice o dati.
- Questo spostamento graduale permette ai sistemi di memoria gerarchica di prevedere quali dati mantenere nella memoria veloce, migliorando la hit ratio e riducendo i tempi di accesso medi.

27.4 Osservazioni

- La località di riferimento è alla base del funzionamento delle **cache** e della gestione efficiente della memoria.
- Esistono due tipi principali di località:
  - **Località temporale**: dati recentemente utilizzati tendono a essere riutilizzati a breve termine.
  - **Località spaziale**: dati vicini tra loro vengono spesso utilizzati in sequenza.
- Sfruttare la località di riferimento permette di progettare memorie veloci con capacità limitata, mantenendo alte prestazioni complessive del sistema.

28 Motivi delle Riferimenti Raggruppati (Reasons for Clustered References)

28.1 Introduzione

Le **riferimenti raggruppati** (clustered references) in memoria spiegano perché la maggior parte degli accessi alla memoria non è casuale, ma concentrata in aree vicine. Questa caratteristica è fondamentale per comprendere l'efficacia della cache e delle strategie di memoria gerarchica.

28.2 Cause principali dei riferimenti raggruppati

- **Esecuzione sequenziale delle istruzioni:** I programmi generalmente eseguono istruzioni in sequenza, con poche deviazioni dovute ai salti condizionali o incondizionati. Pertanto, la prossima istruzione spesso segue immediatamente quella corrente, creando un raggruppamento naturale degli accessi.
- **Loop e ripetizione di istruzioni:** I cicli (*loops*) contengono un piccolo insieme di istruzioni eseguite ripetutamente. Questo porta a riferimenti ripetuti agli stessi indirizzi di memoria, aumentando la località temporale.
- **Strutture dati vicine in memoria:** Le strutture dati come array, liste e record sono memorizzate in locazioni contigue. L'accesso a elementi consecutivi di queste strutture genera riferimenti spazialmente vicini.

28.3 Osservazioni

- Grazie a questi motivi, sia le **istruzioni** sia i **dati** tendono a rimanere **vicini tra loro** durante l'esecuzione di un programma.
- Questo comportamento giustifica l'efficacia delle strategie di caching che trasferiscono blocchi contigui di memoria nella memoria veloce.
- La comprensione dei riferimenti raggruppati è essenziale per progettare sistemi con elevata **hit ratio** e ridurre i tempi medi di accesso alla memoria.

29 Località Spaziale e Temporale (Spatial and Temporal Locality)

29.1 Introduzione

La località di riferimento si suddivide in due concetti fondamentali: **località spaziale** (*spatial locality*) e **località temporale** (*temporal locality*). Questi principi guidano la progettazione di memorie gerarchiche e cache efficienti, massimizzando l'uso della memoria veloce.

29.2 Località Spaziale (Spatial Locality)

- Gli accessi alla memoria avvengono a **locazioni vicine tra loro**.
- Si manifesta durante:
  - Esecuzione sequenziale delle istruzioni.
  - Accesso a **array** o strutture dati contigue.
- Le CPU moderne implementano il **prefetching**, caricando preventivamente nella memoria veloce i dati vicini a quelli richiesti, anticipando gli accessi futuri.

29.3 Località Temporale (Temporal Locality)

- Gli accessi alla memoria si concentrano su **locazioni recentemente utilizzate**.
- Si verifica quando le stesse istruzioni o dati vengono utilizzati ripetutamente, ad esempio all'interno di **loop** o funzioni richiamate frequentemente.

29.4 Obiettivo della gestione della località

- Organizzare il codice e i dati in modo che:
  - La maggior parte degli accessi avvenga in **memoria veloce** e di livello superiore (M_1).
  - Solo una piccola percentuale di accessi raggiunga la memoria più lenta e di livello inferiore (M_2).
  - I **cluster di dati** vengano trasferiti verso i livelli superiori della gerarchia quando necessario.

29.5 Osservazioni

- La combinazione di località spaziale e temporale consente alle memorie veloci di servire la maggior parte degli accessi, aumentando l'efficienza e riducendo il tempo medio di accesso.
- La preallocazione e il movimento dei dati basati su località sono alla base delle politiche di caching, paging e gestione dei buffer nei sistemi moderni.
- Massimizzare la località di riferimento è essenziale per ottenere prestazioni elevate in sistemi multitasking e applicazioni ad alta intensità di memoria.

30 Volatilità e Memoria Cache

30.1 Volatilità

- La **volatilità** indica la proprietà di una memoria di **perdere il contenuto quando l'alimentazione viene interrotta**.
- Esempi di memoria volatile:
  - **Registri** del processore
  - **Cache** della CPU
  - **Memoria principale (RAM)**
- Esempi di memoria non volatile:
  - **Dischi rigidi**, SSD e altre memorie di massa
- La distinzione è cruciale per progettare sistemi che combinano velocità e persistenza dei dati.

30.2 Memoria Cache

- La **cache** è una memoria molto piccola (*pochi MB*) ma **molto veloce**, situata vicino alla CPU.
- Ha il compito di **tenere temporaneamente i dati** trasferiti tra la memoria principale e i registri della CPU.
- In genere, la cache è **trasparente al programmatore e al sistema operativo**, funzionando in modo automatico per migliorare le prestazioni.
- La cache sfrutta la **località temporale** per aumentare l'efficienza: dati recentemente usati sono mantenuti in memoria veloce per accessi futuri.
- I trasferimenti dalla memoria principale alla cache avvengono **per blocchi** (*blocks*), sfruttando la località di riferimento spaziale e temporale.

30.3 Osservazioni

- La memoria cache riduce il tempo medio di accesso alla memoria e aumenta significativamente le prestazioni della CPU.
- La combinazione di registri, cache e memoria principale volatile con memoria di massa non volatile forma la base della gerarchia della memoria.
- La cache è essenziale per sfruttare appieno la velocità del processore senza dover aumentare continuamente la velocità della memoria principale, mantenendo al contempo costi ragionevoli.

31 Memoria Virtuale e Cache del Disco

31.1 Memoria Virtuale

- La **memoria virtuale** funziona come un'estensione della memoria principale, utilizzando lo **spazio di memoria secondaria** (disco) per ospitare dati e istruzioni non immediatamente presenti in RAM.
- Consente ai programmi di **utilizzare più memoria di quanta fisicamente disponibile**, permettendo l'esecuzione di applicazioni più grandi rispetto alla RAM reale.
- Il sistema operativo gestisce la memoria virtuale trasferendo pagine di dati tra disco e RAM in base alle esigenze del programma.

31.2 Cache del Disco

- La **disk cache** è una porzione della memoria principale riservata alle operazioni di lettura e scrittura su disco.
- Migliora le prestazioni del sistema grazie a due meccanismi principali:
  - **Clustering delle scritture:** raggruppa più operazioni di scrittura per effettuare trasferimenti più grandi e meno frequenti, riducendo il numero di accessi al disco.
  - **Lettura anticipata e riuso dei dati:** consente di leggere nuovamente dati già presenti in cache prima che siano scritti definitivamente su disco, migliorando l'efficienza degli accessi.

31.3 Osservazioni

- La combinazione di **memoria virtuale** e **disk cache** consente di gestire grandi quantità di dati senza compromettere le prestazioni.
- La memoria virtuale permette la gestione trasparente di spazi superiori alla RAM fisica, mentre la cache del disco riduce i tempi di accesso e ottimizza le operazioni di I/O.
- Entrambi i concetti sfruttano i principi della **località di riferimento**, trasferendo nella memoria veloce i dati più frequentemente utilizzati per ridurre i tempi di attesa.

32 Gestione della Memoria

32.1 Introduzione

La **gestione della memoria** è uno dei principali servizi forniti dal sistema operativo. Essa garantisce che i programmi possano accedere alla memoria in modo sicuro ed efficiente, massimizzando l'utilizzo delle risorse hardware. La gestione della memoria coinvolge sia la memoria **fisica** sia quella **logica**, formando la base per la **memoria virtuale**.

32.2 Responsabilità principali del sistema operativo nella gestione della memoria

1. **Isolamento dei processi (Process Isolation)**
   - Impedisce ai processi di interferire con i dati o il codice di altri processi.
   - Garantisce che un errore in un programma non comprometta la memoria di altri programmi.
2. **Allocazione e gestione automatica (Automatic Allocation and Management)**
   - Il sistema operativo alloca dinamicamente la memoria ai programmi secondo le necessità.
   - I programmatori non gestiscono direttamente la memoria, riducendo errori come memory leaks o sovrascritture indesiderate.
3. **Supporto alla programmazione modulare (Modular Programming Support)**
   - Permette di suddividere i programmi in **moduli** indipendenti.
   - I moduli possono essere creati, ridimensionati o distrutti dinamicamente durante l'esecuzione del programma.
4. **Protezione e controllo degli accessi (Protection and Access Control)**
   - Consente la condivisione sicura della memoria tra processi.
   - Previene accessi non autorizzati, proteggendo dati critici e garantendo l'integrità del sistema.
5. **Archiviazione a lungo termine (Long-Term Storage)**
   - Fornisce meccanismi per memorizzare dati in modo permanente, tramite il **file system**.
   - La memoria principale funge da ponte tra il processore e lo storage permanente.

32.3 Memoria fisica e logica

- Il sistema operativo considera la memoria sia come:
  - **Memoria fisica**: le locazioni reali presenti nell'hardware (RAM).
  - **Memoria logica**: un'astrazione che permette ai processi di vedere uno spazio di indirizzamento continuo e sicuro.
- Questa distinzione è essenziale per l'implementazione della **memoria virtuale**, che estende la memoria logica oltre i limiti fisici della RAM.

32.4 Osservazioni

- Una gestione efficace della memoria aumenta la **sicurezza**, la **stabilità** e le **prestazioni** del sistema.
- I servizi di memoria del sistema operativo supportano sia il corretto isolamento dei processi sia l'efficiente utilizzo delle risorse hardware disponibili.
- La memoria virtuale, derivata dalla gestione fisica e logica, consente ai programmi di funzionare come se disponessero di uno spazio di indirizzamento continuo e indipendente.

33 Larghezza di Banda della Memoria (Memory Bandwidth)

33.1 Introduzione

Con l'aumento del numero di core e delle dimensioni delle cache nei processori moderni, la velocità della CPU cresce più rapidamente di quella della memoria. Se i dati non si muovono abbastanza velocemente tra memoria e core, alcune unità di elaborazione restano inattive, rallentando l'intero sistema e riducendo i benefici dei processori multicore.

33.2 Problema della saturazione della larghezza di banda

- La CPU può avere molteplici core pronti all'esecuzione, ma se la memoria non fornisce dati sufficientemente rapidi, i core restano **inattivi**.
- Questo fenomeno porta a una riduzione complessiva delle prestazioni, noto come **saturazione della larghezza di banda della memoria**.
- Il tradizionale bus front-side (FSB, *Front-Side Bus*) può diventare un collo di bottiglia, limitando il throughput della memoria verso la CPU.

33.3 Soluzione: NUMA

- **NUMA (Non-Uniform Memory Access)** è un'architettura in cui la memoria è divisa in nodi locali a ciascun processore.
- I core accedono più rapidamente alla memoria locale e più lentamente a quella remota, riducendo la congestione e la saturazione del bus condiviso.
- Questo approccio migliora le prestazioni complessive del sistema, sfruttando meglio i core multipli e le grandi cache.

33.4 Osservazioni

- La gestione efficiente della larghezza di banda della memoria è cruciale nei sistemi multicore e nelle applicazioni ad alta intensità di dati.
- NUMA permette di mantenere elevata la performance dei processori moderni, minimizzando i tempi di attesa legati agli accessi alla memoria.
- Questa strategia è particolarmente utile nei server e nei sistemi ad alte prestazioni, dove la comunicazione tra core e memoria è critica.

34 Piattaforme Applicative (Application Platforms)

34.1 Introduzione

Una **piattaforma applicativa** è la combinazione di **hardware**, **sistema operativo** e **ambiente di sviluppo software** in cui operano sia gli utenti sia gli sviluppatori. Essa definisce l'infrastruttura necessaria per eseguire applicazioni e creare nuovi programmi in modo efficiente e sicuro.

34.2 Esempio di piattaforma applicativa

- Un esempio concreto può essere:
  - Processore Intel Pentium
  - Sistema operativo Windows
  - Ambiente di sviluppo Microsoft Visual Studio ".NET"
- Questa combinazione permette agli sviluppatori di creare applicazioni che gli utenti possono eseguire senza preoccuparsi della gestione diretta dell'hardware.

34.3 Ruolo del Sistema Operativo

- Il **sistema operativo (OS)** fornisce un insieme di funzioni per l'interfaccia con l'utente, organizzate in librerie software chiamate **API (Application Programming Interfaces)**.
- Le API consentono ai programmi di accedere ai servizi del sistema operativo tramite **chiamate di sistema (system calls)**.

34.4 Chiamate di Sistema (System Calls)

- Un **programma utente**, operante nello **User Space**, richiede al sistema operativo, che opera nello **Kernel Space**, di eseguire operazioni specifiche.
- Esempi di operazioni richieste tramite system calls:
  - Accesso a dispositivi hardware
  - Apertura o scrittura di file
  - Allocazione di memoria
- Questo meccanismo garantisce **sicurezza e protezione**, poiché il kernel controlla e autorizza ogni accesso alle risorse hardware.

34.5 Osservazioni

- Le piattaforme applicative forniscono un **astrazione dell'hardware** ai programmatori, semplificando lo sviluppo e la portabilità delle applicazioni.
- L'uso delle API e delle system calls permette di separare chiaramente il livello utente dal livello kernel, riducendo i rischi di errori e garantendo la stabilità del sistema.
- Comprendere il funzionamento delle piattaforme applicative è essenziale per sviluppare software efficiente e sicuro su sistemi moderni.

35 Ambienti Applicativi (Application Environments)

35.1 Sistemi Operativi Generali

- I **sistemi operativi general-purpose** come Windows, Linux e macOS sono progettati per gestire **molteplici applicazioni e compiti contemporaneamente**.
- Questi sistemi richiedono grandi quantità di memoria e spesso sfruttano hardware specializzato, ad esempio per:
  - Audio
  - Video
  - Grafica 3D
- L'obiettivo è fornire un ambiente flessibile e versatile adatto a una vasta gamma di applicazioni e utenti.

35.2 Sistemi Embedded

- I **sistemi embedded**, come quelli presenti nelle smart TV o nei dispositivi IoT, hanno risorse limitate.
- Sono ottimizzati per **funzioni specifiche**, concentrandosi su:
  - Efficienza energetica
  - Affidabilità
  - Stabilità del dispositivo
- L'approccio embedded permette di ottenere alte prestazioni anche con risorse ridotte, adattandosi perfettamente al compito dedicato.

36 Sistemi Real-Time (Real-Time Systems)

36.1 Introduzione

I **sistemi operativi real-time (RTOS)** devono eseguire i compiti entro vincoli temporali rigorosi e prevedibili, chiamati **deadlines**. La gestione dei tempi di esecuzione è fondamentale per garantire il corretto funzionamento in contesti critici.

36.2 Classificazione dei RTOS

- **Hard RTOS:**
  - Le scadenze devono essere sempre rispettate.
  - Esempi: autopilota di aeroplani, unità di controllo automobilistiche.
  - Utilizzati in sistemi **mission-critical** dove ritardi non sono accettabili.
- **Soft RTOS:**
  - Piccoli ritardi occasionali sono tollerati.
  - Esempi: sistemi di database, applicazioni multimediali.
  - Adatti a contesti **business-critical**, dove l'accuratezza dei tempi è importante ma non vitale.

36.3 Osservazioni

- La distinzione tra hard e soft RTOS è cruciale per progettare sistemi con requisiti temporali adeguati all'ambiente operativo.
- I sistemi real-time richiedono una gestione precisa delle risorse e delle priorità, integrando meccanismi di scheduling e gestione delle interruzioni altamente affidabili.
- Comprendere le differenze tra sistemi general-purpose, embedded e real-time è essenziale per scegliere il sistema operativo più adatto alle esigenze di un'applicazione.

37 Virtualizzazione (Virtualization)

37.1 Macchine Virtuali (Virtual Machines)

- Una **macchina virtuale (VM)** è un'astrazione software dell'hardware del computer.
- Spesso viene eseguita sopra un sistema operativo host; un esempio noto è la **Java Virtual Machine**, che permette l'esecuzione di programmi Java indipendentemente dall'hardware sottostante.
- Un **sistema operativo per macchina virtuale (VM-OS)** gestisce le risorse hardware e software virtualizzate attraverso piattaforme di virtualizzazione come:
  - VMware
  - Xen
  - QEMU
- Il VM-OS permette di isolare e gestire risorse come CPU, memoria e dispositivi di I/O, rendendo possibile l'esecuzione simultanea di più ambienti virtuali sullo stesso hardware fisico.

37.2 Ambienti Virtualizzati (Host Domains)

- Gli **host domains** consentono di eseguire **più sistemi operativi guest** contemporaneamente su una singola macchina fisica, supportando la **consolidazione dei server**.
- Permettono anche l'**emulazione**, simulando hardware o software non presente fisicamente sulla macchina host, come nel caso degli emulatori di console di gioco.
- Migliorano la **compatibilità tra piattaforme** e la **portabilità** delle applicazioni, consentendo a programmi sviluppati per un ambiente di funzionare in altri contesti senza modifiche.

37.3 Osservazioni

- La virtualizzazione aumenta l'efficienza dell'hardware, permettendo un utilizzo migliore delle risorse disponibili.
- Consente l'isolamento sicuro dei sistemi operativi e delle applicazioni, riducendo il rischio che un errore in un guest comprometta altri sistemi.
- È una tecnologia chiave per il cloud computing, il testing di software e la gestione flessibile dei data center.

38 Componenti Core di un Sistema Operativo (Core Components of an OS)

38.1 Evoluzione dei Sistemi Operativi

- I primi sistemi informatici non possedevano un **sistema operativo**; i programmi venivano eseguiti direttamente sull'hardware.
- Successivamente sono stati introdotti concetti come **multiprogrammazione** e **time-sharing** per migliorare l'efficienza e l'utilizzo delle risorse hardware.
- L'avvento dei **personal computer** e, più tardi, di Internet ha portato alla nascita di ambienti operativi **veramente distribuiti**.
- Nuovi ambienti e funzionalità hanno trasformato completamente la struttura tradizionale dei sistemi operativi, richiedendo componenti modulari e flessibili.

38.2 Componenti Core di un Sistema Operativo

- Alla base di ogni sistema operativo c'è l'interazione tra l'utente e il sistema, gestita attraverso l'**interfaccia utente (UI)**.
- L'interfaccia utente spesso opera tramite un programma speciale chiamato **Shell** o **interprete dei comandi**, che permette agli utenti di comunicare con il sistema operativo.
- Tipi di interazione con la Shell:
  - **Interfaccia a riga di comando (CLI)**: l'utente invia comandi testuali.
  - **Interfaccia grafica (GUI)**: l'utente interagisce tramite finestre, icone, puntamento, click e trascinamento.
- La **Shell** è separata dal **Kernel**, che costituisce il livello software contenente i componenti core del sistema operativo.

38.3 Osservazioni

- Il **Kernel** gestisce direttamente le risorse hardware, i processi e la memoria, mentre la Shell fornisce un'interfaccia user-friendly.
- Questa separazione tra interfaccia e nucleo del sistema operativo consente flessibilità, sicurezza e la possibilità di evolvere l'UI senza modificare il core.
- Comprendere l'evoluzione storica dei sistemi operativi aiuta a comprendere perché certe strutture, come multiprogrammazione e time-sharing, siano ancora alla base dei moderni OS.

39 Componenti Tipici del Core di un Sistema Operativo (Typical Core Components of an Operating System)

39.1 Componenti Principali

Un sistema operativo moderno include diversi **componenti core** che coordinano le risorse del computer e forniscono servizi essenziali ai programmi. I principali sono:

1. **Process Scheduler (Pianificatore dei Processi)**
   - Decide **quando** e **per quanto tempo** ciascun processo può utilizzare la CPU.
   - Garantisce equità, efficienza e rispetto delle priorità dei processi.
   - Supporta concetti come **multitasking** e **time-sharing**.
2. **Memory Manager (Gestore della Memoria)**
   - Controlla quanta memoria viene allocata a ciascun processo e per quanto tempo.
   - Implementa meccanismi di protezione e isolamento dei processi.
   - Supporta la memoria virtuale, la gestione delle cache e la paginazione.
3. **I/O Device Manager (Gestore dei Dispositivi di I/O)**
   - Gestisce la comunicazione tra il sistema e i dispositivi esterni.
   - Coordina trasferimenti di dati, bufferizzazione e gestione delle interruzioni.
   - Permette ai programmi di accedere ai dispositivi in modo uniforme tramite driver.
4. **Interprocess Communication (IPC) Manager**
   - Gestisce la comunicazione e la sincronizzazione tra processi.
   - Supporta meccanismi come code, semafori, mutex, pipe e socket.
   - Essenziale per applicazioni multi-processo e sistemi distribuiti.
5. **File System Manager (Gestore del File System)**
   - Organizza i dati memorizzati e fornisce un'interfaccia per accedere a file e directory.
   - Gestisce operazioni come lettura, scrittura, creazione, cancellazione e permessi di accesso.
   - Garantisce integrità dei dati e gestione efficiente dello spazio di archiviazione.

39.2 Osservazioni

- Questi componenti core lavorano insieme per garantire che i processi siano eseguiti correttamente, le risorse siano utilizzate efficientemente e l'utente abbia un accesso sicuro e coerente ai dati.
- Ogni componente è strettamente integrato con gli altri: ad esempio, il memory manager collabora con il process scheduler per allocare memoria ai processi in esecuzione.
- La modularità dei componenti core permette aggiornamenti e miglioramenti senza compromettere l'intero sistema operativo.

40 Importanza della Multiprogrammazione (Importance of Multiprogramming)

40.1 Introduzione

La **multiprogrammazione** è una tecnica fondamentale per aumentare l'efficienza dei sistemi operativi moderni. Permette di sfruttare al massimo le risorse hardware, come la CPU e i dispositivi di I/O, evitando tempi morti durante l'esecuzione dei programmi.

40.2 Concetti chiave

- Un singolo utente o processo non può occupare completamente sia la CPU sia i dispositivi di I/O allo stesso tempo. Senza multiprogrammazione, parte delle risorse rimarrebbe inattiva.
- La multiprogrammazione consiste nel mantenere **più programmi (job)** in memoria principale simultaneamente.
- In questo modo, mentre un job è in attesa di completare operazioni di I/O, la CPU può eseguire un altro job, evitando sprechi di tempo e migliorando le prestazioni complessive del sistema.

40.3 Scheduling dei Job

- Il job che ottiene il tempo CPU viene selezionato mediante un **algoritmo di scheduling dei job**.
- Quando un job entra in stato di attesa (ad esempio, per operazioni di input/output), il sistema operativo sceglie un altro job pronto per l'esecuzione.
- Questo approccio permette di mantenere la CPU **attiva quasi continuamente**, aumentando il throughput del sistema e riducendo i tempi di inattività.

40.4 Osservazioni

- La multiprogrammazione è alla base del **multitasking** nei moderni sistemi operativi, consentendo la gestione simultanea di più processi.
- Migliora l'efficienza complessiva del sistema, bilanciando il carico tra CPU e dispositivi di I/O.
- È un principio fondamentale anche per l'implementazione di sistemi real-time e server multitasking, dove la continuità e la reattività del servizio sono critiche.

41 Importanza del Multitasking (Importance of Multitasking)

41.1 Introduzione

Il **timesharing**, noto anche come **multitasking**, rappresenta l'estensione logica della multiprogrammazione. Permette a più utenti o processi di condividere la CPU in modo rapido e apparentemente simultaneo, migliorando l'interattività e l'efficienza del sistema.

41.2 Caratteristiche principali

- La CPU **cambia rapidamente tra i job**, permettendo agli utenti di interagire con i loro programmi mentre questi sono in esecuzione. Questo concetto prende il nome di **interactive computing**.
- Il **tempo di risposta** per un'azione utente dovrebbe essere inferiore a un secondo, garantendo un'esperienza fluida e reattiva.
- Ogni utente ha almeno un programma in memoria, noto come **processo (process)**.

41.3 Scheduling della CPU

- Quando più job sono pronti contemporaneamente, il sistema operativo utilizza un **algoritmo di scheduling della CPU** per decidere quale processo eseguire per primo.
- Lo scheduling determina l'ordine e la durata dei time slice assegnati ai processi, garantendo equità e rispetto delle priorità.

41.4 Swapping della Memoria

- Se la memoria principale è piena, i processi inattivi vengono temporaneamente **spostati dentro e fuori dalla memoria principale**, mediante un meccanismo chiamato **swapping**.
- Lo swapping consente di mantenere attivi più processi anche quando lo spazio fisico di RAM è limitato, sfruttando la memoria secondaria per conservare temporaneamente i dati dei processi sospesi.

41.5 Osservazioni

- Il multitasking aumenta notevolmente la **responsività del sistema** e l'efficienza dell'uso della CPU.
- Combinato con la multiprogrammazione, permette di gestire grandi carichi di lavoro, ottimizzando l'esecuzione simultanea di più processi.
- La capacità di spostare processi tra memoria principale e secondaria (swapping) è essenziale per garantire la continuità del servizio in sistemi con risorse limitate.

42 Gestione degli Errori nel Sistema Operativo: Modalità Doppia (Error Handling in the OS: Dual Mode)

42.1 Introduzione

Oltre ai guasti hardware (**faults**), gli errori software o le richieste di servizio possono generare **eccezioni** (o **traps**). La gestione corretta di questi eventi è essenziale per garantire la stabilità e la sicurezza del sistema operativo.

42.2 Esempi di Errori ed Eccezioni

- Divisione per zero
- Accesso a locazioni di memoria non valide
- Errori nelle richieste di I/O
- Tentativi dei processi di modificare aree di memoria riservate al sistema operativo o ad altri processi

42.3 Modalità Doppia (Dual Mode)

- La **modalità doppia** permette al sistema operativo di proteggere sé stesso e gli altri componenti critici del sistema.
- Esistono due modalità operative principali:
  1. **User Mode (Modalità Utente)**: modalità in cui i processi utente vengono eseguiti con privilegi limitati, impedendo l'accesso diretto alle risorse critiche del sistema.
  2. **Kernel Mode (Modalità Kernel)**: modalità privilegiata in cui il sistema operativo può accedere liberamente all'hardware e alle strutture di memoria protette.

42.4 Osservazioni

- La separazione tra User Mode e Kernel Mode è fondamentale per la sicurezza e la stabilità del sistema.
- Ogni eccezione o trap generata in User Mode trasferisce il controllo al Kernel Mode, dove il sistema operativo può gestire correttamente l'errore.
- Questo meccanismo protegge sia il kernel sia i processi utente, prevenendo comportamenti indesiderati come corruzione della memoria o accessi non autorizzati alle risorse hardware.

43 Gestione della Modalità Doppia (Dual-Mode Management)

43.1 Introduzione

La **modalità doppia** è gestita da un **bit di modalità** incorporato direttamente nell'hardware del processore. Questo meccanismo permette al sistema operativo di distinguere tra esecuzione di codice utente e codice kernel, garantendo sicurezza e protezione delle risorse critiche.

43.2 Funzionamento del Bit di Modalità

- Il valore del **bit di modalità** determina se il sistema sta eseguendo codice in **user mode** o **kernel mode**.
- Solo in **kernel mode** è possibile eseguire istruzioni privilegiate, necessarie per gestire direttamente l'hardware o modificare strutture di sistema protette.

43.3 Transizione tra Modalità

- Una **system call** permette di passare dalla modalità utente alla modalità kernel.
- Durante la system call, il sistema operativo prende il controllo per eseguire operazioni privilegiate richieste dal programma utente.
- Al termine della system call, il controllo ritorna automaticamente alla **modalità utente**, permettendo al programma di continuare l'esecuzione senza rischi per la sicurezza del sistema.

43.4 Osservazioni

- Il meccanismo di dual-mode è essenziale per **protezione del kernel**, **isolamento dei processi** e corretta gestione degli errori.
- Grazie al bit di modalità, il processore può prevenire l'esecuzione di istruzioni critiche da parte di codice non autorizzato, riducendo il rischio di crash o corruzione della memoria.

44 Gestione dei Processi (Process Management)

44.1 Definizione di Processo

- Un **processo** è un **programma in esecuzione**.
- A differenza di un programma, che è un'entità passiva, un processo è un'entità **attiva**.
- Il processo costituisce l'**unità di lavoro fondamentale** all'interno di un sistema operativo.

44.2 Risorse Necessarie a un Processo

- Un processo necessita di diverse risorse per eseguire correttamente il proprio compito:
  - Tempo CPU
  - Memoria
  - File
  - Dispositivi di input/output (I/O)
  - Dati iniziali
- Al termine del processo, tutte le risorse riutilizzabili precedentemente assegnate vengono liberate dal sistema operativo.

44.3 Processi Single-Thread e Multi-Thread

- **Processi single-thread**:
  - Hanno un solo **program counter** che indica la locazione della prossima istruzione da eseguire.
  - Le istruzioni vengono eseguite **sequenzialmente**, una dopo l'altra, fino al termine del processo.
- **Processi multi-thread**:
  - Ogni thread ha il proprio **program counter**.
  - Permettono a più parti dello stesso programma di essere eseguite **simultaneamente**, sfruttando meglio la CPU e le risorse disponibili.

44.4 Esecuzione Concurrente dei Processi

- Un sistema operativo esegue generalmente molti processi contemporaneamente:
  - **User processes** (processi utente)
  - **System processes** (processi di sistema, chiamati *daemons* in Unix o *services* in Windows)
- La **concorrenza** viene realizzata tramite la **multiplexing della CPU**, in cui la CPU passa rapidamente da un processo o thread all'altro.

44.5 Responsabilità del Sistema Operativo nella Gestione dei Processi

- Creazione e eliminazione di processi utente e di sistema.
- Sospensione e ripresa dei processi.
- Fornitura di meccanismi per:
  - **Sincronizzazione dei processi**
  - **Comunicazione tra processi (IPC)**
  - **Gestione dei deadlock**

44.6 Osservazioni

- La gestione dei processi è cruciale per garantire l'efficienza, la sicurezza e la stabilità del sistema operativo.
- Il supporto ai processi multi-thread e alla concorrenza permette di sfruttare appieno le architetture multicore e le risorse di sistema.
- La sincronizzazione e la comunicazione tra processi prevengono condizioni di competizione e deadlock, assicurando l'integrità dei dati e la correttezza delle operazioni.

45 Gestione della Memoria Principale (Main Memory Management)

45.1 Introduzione

La **memoria principale** è un'area di archiviazione di grandi dimensioni, suddivisa in unità ciascuna con un **indirizzo univoco** (misurato in GB). Essa costituisce una memoria ad accesso rapido condivisa tra CPU e dispositivi di I/O.

- La CPU legge istruzioni e dati dalla memoria durante la fase di **fetch** e può scrivere dati in memoria quando necessario.
- Il modulo di gestione della memoria monitora costantemente cosa è memorizzato in memoria per ottimizzare l'uso della CPU e i tempi di risposta del sistema.

45.2 Responsabilità del Sistema Operativo nella Gestione della Memoria

Il sistema operativo è responsabile di diverse attività critiche relative alla memoria principale:

- **Tracciamento della memoria**: tenere traccia delle aree di memoria in uso e dei processi che le occupano.
- **Caricamento dei processi**: decidere quali processi caricare in memoria quando lo spazio diventa disponibile.
- **Allocazione e deallocazione**: assegnare e liberare lo spazio di memoria secondo le necessità dei programmi.

46 Gestione della Memoria Secondaria (Secondary Memory Management)

46.1 Introduzione

La memoria secondaria fornisce uno spazio di archiviazione a lungo termine. Il sistema operativo offre una **visione logica e uniforme** dei dati memorizzati, nascondendo le caratteristiche fisiche dei dispositivi di archiviazione.

- Il concetto chiave introdotto dal sistema operativo è quello di **file**, ovvero un'unità logica che rappresenta un insieme di dati memorizzati.
- Un file è un'**astrazione indipendente** dal supporto fisico utilizzato, permettendo agli utenti e ai programmi di lavorare con i dati senza preoccuparsi delle specifiche hardware.

46.2 Gestione dei Dispositivi di Memoria

- Ogni dispositivo di memorizzazione è gestito da un **device driver**, che nasconde i dettagli hardware, come:
  - Metodi di accesso
  - Velocità del dispositivo
  - Capacità totale
  - Velocità di trasferimento dati
- Il device driver fornisce un'interfaccia uniforme al sistema operativo, permettendo la gestione dei file e delle operazioni di lettura/scrittura in modo coerente.

46.3 Osservazioni

- La gestione della memoria principale e secondaria è essenziale per l'efficienza complessiva del sistema.
- La distinzione tra memoria veloce (RAM) e memoria secondaria (dischi) permette di combinare prestazioni elevate e capacità di archiviazione a lungo termine.
- L'astrazione dei file e l'uso dei device driver consentono al sistema operativo di garantire portabilità, sicurezza e uniformità nella gestione dei dati.

47 Gestione del File System (File System Management)

47.1 Introduzione

Un **file** è una raccolta di informazioni correlate, che può contenere programmi o dati. I file sono organizzati in **directory**, creando una struttura gerarchica che facilita l'accesso e la gestione.

- Il sistema operativo utilizza **controlli di accesso** per determinare quali utenti possono leggere, scrivere o modificare specifici file.
- L'OS fornisce un'interfaccia uniforme per la gestione dei file, indipendentemente dal dispositivo di memorizzazione fisico.

47.2 Responsabilità del Sistema Operativo nella Gestione dei File

Il sistema operativo svolge diverse attività chiave relative alla gestione dei file:

- **Creazione e cancellazione**: permette di creare e rimuovere file e directory secondo le necessità dell'utente o delle applicazioni.
- **Operazioni di base sui file**: fornisce funzioni per leggere, scrivere e modificare i contenuti dei file.
- **Collegamento ai dispositivi di memorizzazione secondaria**: gestisce il mapping dei file ai supporti fisici come dischi rigidi, SSD o altri dispositivi di archiviazione.
- **Backup**: effettua copie di sicurezza su dispositivi di archiviazione stabili per garantire la protezione dei dati.

47.3 Osservazioni

- La gestione dei file è essenziale per organizzare i dati e fornire un accesso sicuro e controllato alle informazioni.
- I controlli di accesso prevengono operazioni non autorizzate e proteggono l'integrità dei dati.
- La separazione tra struttura logica (file e directory) e supporto fisico (dispositivo di archiviazione) rende il file system flessibile e portabile tra diverse piattaforme hardware.

48 Gestione dei Dischi (Disk Management)

48.1 Introduzione

La **memoria principale** è volatile e di capacità limitata, quindi non può essere utilizzata per la conservazione permanente dei dati. Per questo motivo, il sistema deve utilizzare la **memoria secondaria** per salvare i dati provenienti dalla RAM.

- Nei sistemi moderni, i **dischi** (HDD o SSD) rappresentano il principale mezzo di archiviazione.
- L'efficienza nella gestione della memoria secondaria è fondamentale per le prestazioni complessive del sistema, poiché queste dipendono in larga misura dalla velocità delle operazioni su disco.

48.2 Responsabilità del Sistema Operativo nella Gestione dei Dischi

Il sistema operativo svolge diverse funzioni chiave nella gestione dei dischi:

- **Gestione dello spazio libero**: tiene traccia delle aree non utilizzate del disco.
- **Allocazione dello spazio**: assegna blocchi di memoria secondaria ai file e ai processi secondo necessità.
- **Scheduling degli accessi al disco**: organizza le operazioni di lettura e scrittura per migliorare le prestazioni e ridurre i tempi di attesa.

48.3 Osservazioni

- Poiché le operazioni su disco sono significativamente più lente rispetto all'accesso alla memoria principale, una gestione efficiente è essenziale per evitare colli di bottiglia.
- Gli algoritmi di disk scheduling contribuiscono a ottimizzare i movimenti della testina (nei dischi meccanici) o la gestione delle richieste di I/O.
- Una buona gestione del disco migliora la velocità di accesso ai dati, l'affidabilità e le prestazioni complessive del sistema.

49 Gestione della Memoria Terziaria (Tertiary Memory Management)

49.1 Introduzione

Alcuni dispositivi di archiviazione di massa, utilizzati principalmente per il **backup dei dati** e spesso mantenuti offline, non richiedono velocità elevate di accesso. Questa categoria prende il nome di **memoria terziaria**.

- Include dispositivi come:
  - Dischi ottici
  - Nastri magnetici
- Sebbene siano più lenti rispetto ai dischi principali, devono comunque essere gestiti in modo efficiente dal sistema operativo.

49.2 Caratteristiche dei Dispositivi di Memoria Terziaria

I dispositivi di memoria terziaria possono essere classificati in base alle modalità di scrittura:

- **WORM (Write Once, Read Many)**:
  - I dati possono essere scritti una sola volta.
  - Possono essere letti molte volte senza modifiche.
  - Utilizzati per archiviazione permanente o dati che non devono essere alterati.
- **RW (Read and Write)**:
  - Permettono sia la lettura sia la scrittura dei dati più volte.
  - Offrono maggiore flessibilità nella gestione delle informazioni archiviate.

49.3 Osservazioni

- La memoria terziaria è fondamentale per strategie di **backup** e **archiviazione a lungo termine**.
- Anche se la velocità non è prioritaria, l'organizzazione, l'affidabilità e la corretta gestione dello spazio rimangono aspetti essenziali.
- Il sistema operativo deve fornire meccanismi per l'integrazione di questi dispositivi nel sistema di gestione dei file e delle operazioni di archiviazione.

50 Gestione dei Dispositivi di I/O (I/O Device Management)

50.1 Introduzione

Uno degli obiettivi principali del sistema operativo è **nascondere i dettagli hardware** all'utente e ai programmi applicativi, fornendo un'interfaccia semplice e uniforme per l'uso dei dispositivi di input/output.

50.2 Struttura del Sottosistema di I/O

Il sottosistema di I/O è composto da diversi elementi fondamentali:

- **Componente di gestione della memoria**, che include:
  - **Buffering**: memorizzazione temporanea dei dati nella memoria locale del dispositivo durante il trasferimento, per compensare differenze di velocità tra CPU e dispositivi.
  - **Caching**: memorizzazione parziale dei dati più frequentemente utilizzati in una memoria ad accesso rapido, per migliorare le prestazioni.
  - **Spooling**: trasferimento dei dati tra diverse aree di memoria per supportare la memoria virtuale e le operazioni di I/O in coda (ad esempio, nella gestione delle stampanti).
- **Interfaccia generale per i device driver**: fornisce un meccanismo standardizzato attraverso cui il sistema operativo comunica con i dispositivi hardware.
- **Driver specifici per ogni dispositivo**: ogni componente hardware (stampante, disco, tastiera, scheda di rete, ecc.) è gestito da un driver dedicato che ne controlla il funzionamento e ne nasconde le caratteristiche tecniche.

50.3 Osservazioni

- Il sottosistema di I/O migliora l'efficienza del sistema riducendo i tempi di attesa e ottimizzando i trasferimenti di dati.
- L'astrazione fornita dai driver consente portabilità e modularità: nuovi dispositivi possono essere aggiunti senza modificare il resto del sistema operativo.
- Buffering, caching e spooling sono tecniche fondamentali per bilanciare le differenze di velocità tra CPU, memoria e dispositivi di I/O.

51 Protezione e Sicurezza (Protection and Security)

51.1 Protezione

- La **protezione** è il meccanismo utilizzato per controllare l'accesso di utenti o processi alle risorse del sistema.
- Definisce **chi può fare cosa** su risorse come file, memoria, dispositivi e processi.
- Si basa su politiche di accesso e controlli implementati dal sistema operativo.

51.2 Sicurezza

- La **sicurezza** comprende i meccanismi adottati per difendere il sistema da attacchi interni ed esterni.
- Esempi di minacce:
  - Denial-of-Service (DoS)
  - Worm
  - Virus
  - Hacker
- L'obiettivo è garantire **riservatezza, integrità e disponibilità** delle risorse del sistema.

51.3 Identificazione e Controllo degli Accessi

- Il sistema identifica prima gli utenti per determinare quali risorse possono essere accessibili.
- Ogni utente possiede un **User ID (UID)**, composto da:
  - Un nome
  - Un numero univoco
- Il UID garantisce la corretta proprietà e gestione di file e processi.
- Un **Group ID (GID)** permette a più utenti di condividere l'accesso a risorse comuni, come file e processi.

51.4 Osservazioni

- La protezione riguarda il controllo interno degli accessi, mentre la sicurezza si concentra sulla difesa contro minacce e attacchi.
- Meccanismi come autenticazione, autorizzazione e crittografia rafforzano la sicurezza del sistema.
- Una corretta gestione di UID e GID è fondamentale nei sistemi multiutente per garantire isolamento e condivisione controllata delle risorse.

52 Ambienti Multiprogrammati (Multiprogrammed Environments)

52.1 Introduzione

I sistemi moderni utilizzano la **multiprogrammazione**, in cui più applicazioni vengono eseguite in modo concorrente. Questo consente un uso più efficiente della CPU e delle risorse di sistema.

52.2 Gestione dei Processi e dei Thread

- Il **kernel** gestisce l'esecuzione concorrente dei **processi**, che sono le istanze attive dei programmi memorizzati nella memoria principale.
- Parti di un processo che possono essere eseguite indipendentemente ma che condividono lo stesso spazio di memoria sono chiamate **thread**.
- I thread permettono un'esecuzione più efficiente e parallela all'interno dello stesso processo.

52.3 Accesso ai Dispositivi di I/O

- Per accedere ai dispositivi di input/output, i processi devono effettuare **system call** al sistema operativo.
- Le system call vengono gestite dai **device driver**, che controllano direttamente l'hardware.
- I device driver sono componenti software che traducono i comandi del sistema operativo in azioni specifiche per l'hardware.

52.4 Osservazioni

- La multiprogrammazione migliora il throughput del sistema mantenendo la CPU attiva mentre altri processi attendono operazioni di I/O.
- La separazione tra kernel, processi, thread e driver garantisce modularità e sicurezza.
- I driver permettono al sistema operativo di supportare diversi dispositivi senza modificare le applicazioni utente.

53 Ambienti di Personal Computing (Personal Computing Environments)

53.1 Evoluzione degli Ambienti di Elaborazione

L'elaborazione tradizionale si è evoluta nel tempo e i confini tra i diversi ambienti di calcolo sono diventati meno definiti. Oggi, sistemi domestici, aziendali e distribuiti condividono tecnologie comuni come reti ad alta velocità, servizi cloud e accesso remoto.

53.2 Office Automation

53.2.1 In passato

- Più PC collegati in una rete locale (LAN).
- Presenza di server dedicati per:
  - Condivisione di file
  - Gestione delle stampanti

53.2.2 Oggi

- Utilizzo di portali online che permettono l'accesso via Internet ai server interni.
- Accesso remoto a sistemi aziendali.
- Integrazione con servizi cloud e strumenti collaborativi.

53.3 Home Computing

53.3.1 In passato

- Un singolo PC collegato a Internet tramite modem.

53.3.2 Oggi

- Piccole reti locali domestiche (home LAN).
- Connessioni Internet ad alta velocità (fibra, ADSL, 5G).
- Presenza di firewall e sistemi di sicurezza per prevenire intrusioni.
- Integrazione di dispositivi smart (IoT), come smart TV, assistenti vocali e sistemi di videosorveglianza.

53.4 Osservazioni

- Gli ambienti di personal computing moderni combinano caratteristiche di sistemi distribuiti, embedded e cloud.
- La sicurezza e la connettività sono diventate aspetti centrali sia in ambito aziendale sia domestico.
- L'evoluzione tecnologica ha reso più sfumata la distinzione tra ambienti professionali e personali.

54 Ambienti di Calcolo Distribuito (Distributed Computing Environments)

54.1 Definizione di Sistema Distribuito

Un **sistema distribuito** è un insieme di processori che:

- Non condividono memoria.
- Non condividono un clock globale.
- Dispongono ciascuno di una propria memoria locale.

I processori sono collegati tra loro tramite una **rete di comunicazione**, che consente lo scambio di informazioni tra i nodi del sistema.

54.2 Comunicazione

- La comunicazione tra i processori avviene secondo uno **specifico protocollo**.
- I protocolli definiscono le regole per la trasmissione, ricezione e interpretazione dei dati.
- La cooperazione tra nodi avviene tramite messaggi scambiati attraverso la rete.

54.3 Vantaggi dei Sistemi Distribuiti

Un sistema distribuito consente agli utenti di accedere a **risorse condivise**, offrendo diversi benefici:

- **Maggiore velocità di elaborazione**: grazie alla suddivisione dei compiti tra più processori.
- **Maggiore disponibilità dei dati**: le informazioni possono essere replicate su più nodi.
- **Maggiore affidabilità**: se un nodo fallisce, altri possono continuare a operare.

54.4 Osservazioni

- I sistemi distribuiti sono alla base di tecnologie moderne come il cloud computing e i data center.
- L'assenza di memoria condivisa richiede meccanismi efficienti di comunicazione e sincronizzazione.
- La progettazione di un sistema distribuito deve considerare aspetti come latenza di rete, tolleranza ai guasti e consistenza dei dati.

55 Ambienti Distribuiti in Rete (Networked Distributed Environments)

55.1 Modello Client–Server

Nel modello **Client–Server**, i tradizionali terminali collegati a mainframe sono stati sostituiti da **PC (client)** e **server** dedicati.

- I **client** richiedono servizi.
- I **server** forniscono accesso a servizi e risorse condivise.

Questo modello rappresenta una forma comune di sistema distribuito in cui le funzionalità sono suddivise tra macchine diverse collegate tramite rete.

55.2 Servizi Offerti dai Server

I server possono fornire diversi tipi di servizi, tra cui:

- **Stampa** (print server)
- **Archiviazione e recupero di file** (file server)
- **Gestione di database** (database server)
- **Accesso a Internet** (proxy o gateway server)

55.3 Caratteristiche del Modello Client–Server

- Centralizzazione delle risorse e della gestione.
- Maggiore controllo su sicurezza e accessi.
- Scalabilità: nuovi client possono essere aggiunti facilmente alla rete.

55.4 Osservazioni

- Il modello client–server è alla base di molte applicazioni moderne, incluse applicazioni web e servizi cloud.
- La separazione tra richiesta (client) e fornitura del servizio (server) consente una gestione più efficiente delle risorse.
- È un'architettura fondamentale nei sistemi distribuiti in rete.

56 Modello Peer-to-Peer (Peer-to-Peer Model)

56.1 Introduzione

Il **modello peer-to-peer (P2P)** rappresenta un tipo differente di sistema distribuito rispetto al tradizionale client–server.

- Nel P2P non esiste una distinzione netta tra client e server.
- Il sistema è costituito da **nodi equivalenti** detti **peer**.
- Ogni peer può comportarsi sia come client che come server verso altri nodi della rete.

56.2 Caratteristiche del Modello P2P

- **Decentralizzazione**: nessun nodo centrale gestisce le risorse.
- **Condivisione diretta**: ogni peer può fornire o richiedere risorse, come file o dati.
- **Scalabilità**: nuovi peer possono unirsi o lasciare la rete senza influire sull'intero sistema.

56.3 Esempi Famosi

- **Napster**: uno dei primi sistemi P2P per la condivisione di file musicali.
- **Gnutella**: rete P2P decentralizzata per la condivisione di file generici.

56.4 Osservazioni

- Il modello P2P è utile per la distribuzione di contenuti senza dipendere da server centrali.
- La gestione dei peer e la tolleranza ai guasti sono sfide importanti in reti P2P.
- A differenza del modello client–server, il P2P può aumentare la resilienza e ridurre i colli di bottiglia legati ai server centrali.

57 Obiettivi di un Sistema Operativo (Objectives of an Operating System)

Gli utenti si aspettano che un sistema operativo soddisfi diverse proprietà e obiettivi fondamentali, tra cui:

- **Efficienza (Efficiency)**: utilizzare le risorse del sistema in modo ottimale, minimizzando i tempi di inattività della CPU e dei dispositivi di I/O.
- **Robustezza (Robustness)**: mantenere il sistema stabile anche in presenza di errori hardware, software o input non validi.
- **Scalabilità (Scalability)**: permettere l'espansione del sistema aggiungendo più utenti, processori o dispositivi senza degradare le prestazioni.
- **Estendibilità (Extensibility)**: consentire l'aggiunta di nuove funzionalità senza modificare l'intera architettura del sistema.
- **Portabilità (Portability)**: facilitare l'esecuzione del sistema operativo su diverse piattaforme hardware.
- **Sicurezza (Security)**: proteggere il sistema da accessi non autorizzati e minacce esterne.
- **Protezione (Protection)**: garantire che processi e utenti accedano solo alle risorse a cui sono autorizzati.
- **Interattività (Interactivity)**: consentire un'interazione efficiente e rapida tra utente e sistema.
- **Usabilità (Usability)**: rendere il sistema comprensibile e semplice da usare per l'utente finale.

58 Servizi del Sistema Operativo (Operating System Services)

Il sistema operativo offre diversi servizi chiave, indispensabili per il funzionamento del computer e per l'interazione con l'utente:

- **Interfaccia Utente (User Interface)**: può essere grafica (GUI), basata su linea di comando (CLI) o touchscreen. Permette all'utente di comunicare con il sistema operativo.
- **Esecuzione dei Programmi (Program Execution)**: carica i programmi in memoria, gestisce la loro esecuzione e la terminazione al completamento delle operazioni.
- **Operazioni di I/O (I/O Operations)**: gestisce l'accesso sicuro ed efficiente ai dispositivi di input/output e ai file.

- **Gestione del File System (File System Management)**:
  - Crea, cancella, legge, scrive e ricerca file e directory.
  - Gestisce i permessi di accesso ai file e supporta diversi tipi di file system.
- **Comunicazione (Communication)**:
  - Permette lo scambio di informazioni tra processi nello stesso sistema o su sistemi collegati in rete.
  - Può utilizzare **memoria condivisa** o **passaggio di messaggi** per la comunicazione tra processi.
- **Rilevamento degli Errori (Error Detection)**:
  - Individua e gestisce errori hardware, software e operativi generati dall'utente.
  - Garantisce il funzionamento corretto e sicuro del sistema.

58.1 Osservazioni

- I servizi forniti dal sistema operativo costituiscono l'interfaccia tra l'utente, i programmi applicativi e l'hardware.
- La corretta implementazione di questi servizi è essenziale per garantire efficienza, sicurezza e usabilità del sistema.
- Ogni servizio è progettato per astrarre complessità hardware e fornire un ambiente coerente e affidabile per l'esecuzione dei programmi.

59 Servizi per l'Efficienza del Sistema (System Efficiency Services)

Il sistema operativo fornisce servizi mirati a ottimizzare le prestazioni complessive e garantire un utilizzo corretto delle risorse.

59.1 Allocazione delle Risorse (Resource Allocation)

- Distribuisce le risorse del sistema, come:
  - Tempo di CPU
  - Memoria principale
  - Spazio di archiviazione
  - Dispositivi di input/output

  tra i processi in esecuzione.
- Utilizza **algoritmi di scheduling della CPU** per massimizzare sia le prestazioni che l'equità tra processi.
- Obiettivo: garantire che tutte le applicazioni ottengano le risorse necessarie senza sprechi.

59.2 Registrazione delle Attività (Logging)

- Tiene traccia delle attività dei programmi e dell'utilizzo delle risorse.
- Utile per:
  - Contabilità del sistema (system accounting)
  - Statistiche sulle prestazioni
  - Analisi di errori o anomalie
- Permette agli amministratori di ottimizzare il sistema e prevenire problemi futuri.

59.3 Protezione e Sicurezza (Protection and Security)

- Garantisce accessi controllati alle risorse del sistema.
- Richiede **autenticazione degli utenti** e monitora accessi non autorizzati.
- Previene interferenze tra processi o tra processi e il sistema operativo.
- Contribuisce a mantenere l'integrità e la stabilità del sistema.

59.4 Osservazioni

- L'allocazione efficiente delle risorse è essenziale per il bilanciamento del carico e il throughput del sistema.
- Il logging fornisce dati fondamentali per la diagnostica e il miglioramento continuo.
- Protezione e sicurezza sono complementari: proteggono il sistema sia da errori accidentali sia da accessi malevoli.

60 Shell e Esecuzione dei Comandi (Shells and Command Execution)

60.1 Introduzione

Il **command interpreter**, comunemente chiamato **shell**, viene avviato al momento del login dell'utente o all'inizio di un job. La shell costituisce l'interfaccia principale tra l'utente e il sistema operativo, permettendo l'interazione tramite comandi testuali.

- Gli utenti possono scegliere tra diverse shell in base alle proprie preferenze e necessità.
- La shell interpreta i comandi dell'utente e li esegue, fornendo un ambiente di lavoro coerente.

60.2 Ruolo Principale (Main Role)

- Legge e interpreta i comandi inseriti dall'utente.
- Esegue le operazioni richieste, che includono attività comuni come:
  - Creazione e cancellazione di file o directory
  - Elencazione di file e directory
  - Copia di file
  - Esecuzione di programmi o script

60.3 Metodi di Esecuzione dei Comandi (Command Execution Methods)

60.3.1 Comandi Integrati (Built-in Commands)

- La shell contiene direttamente il codice per ogni comando integrato.
- La dimensione della shell dipende dal numero di comandi implementati al suo interno.
- Vantaggio: esecuzione rapida dei comandi senza avviare processi esterni.

60.3.2 Comandi Esterni (External Commands)

- Tipico esempio: sistemi UNIX.
- I comandi sono programmi separati, ad esempio `rm file.txt`.
- La shell carica il programma corrispondente ed esegue l'operazione con i parametri forniti.
- Vantaggio: facilità di estensione del sistema aggiungendo nuovi eseguibili senza modificare la shell stessa.

60.4 Osservazioni

- La distinzione tra comandi integrati ed esterni permette di bilanciare efficienza e flessibilità.
- Le shell moderne offrono funzionalità aggiuntive come scripting, gestione di variabili e piping, migliorando l'interattività e l'automazione.
- Comprendere il funzionamento della shell è fondamentale per la gestione del sistema e per l'uso efficace di un sistema operativo multiutente.

61 Interfaccia Grafica Utente (Graphical User Interface)

61.1 Introduzione

L'**Interfaccia Grafica Utente (GUI)** rappresenta un modo intuitivo e user-friendly per interagire con il sistema operativo, evitando la necessità di digitare comandi direttamente. La GUI sfrutta elementi visivi e interattivi per facilitare l'uso del computer anche agli utenti non esperti.

61.2 Caratteristiche Chiave (Key Features)

- **Finestre e Menu**: elementi navigabili tramite mouse che organizzano programmi e funzioni in spazi distinti.
- **Metafora del Desktop**: icone rappresentano programmi, file e directory, simulando un ambiente di lavoro reale.
- **Puntatore del Mouse**: utilizzato per selezionare oggetti e aprire applicazioni.

61.3 Azioni dell'Utente (User Actions)

- Muovere il puntatore del mouse per scegliere icone o elementi visivi.
- Cliccare su un'icona per lanciare un programma o selezionare file e cartelle.

61.4 Osservazioni

- La GUI riduce la curva di apprendimento per nuovi utenti rispetto alle interfacce a linea di comando.
- Favorisce l'interattività, la multimedialità e l'uso intuitivo di applicazioni complesse.
- Gli elementi grafici possono essere combinati con strumenti di accessibilità per utenti con disabilità, aumentando l'inclusività del sistema operativo.

62 Interazione Touchscreen nei Sistemi Mobili (Touchscreen Interaction in Mobile Systems)

62.1 Introduzione

Le tradizionali interfacce basate su **linea di comando** o su **mouse e tastiera** non sono pratiche per i dispositivi mobili. Per questo motivo, smartphone e tablet utilizzano **schermi touchscreen** come principale metodo di interazione con il sistema operativo.

62.2 Idea Principale

- Gli utenti interagiscono direttamente con gli elementi visualizzati sullo schermo.
- Le azioni vengono eseguite tramite **gesti sullo schermo**.

62.3 Gesti Comuni

- **Tap**: toccare lo schermo per selezionare un elemento o aprire un'applicazione.
- **Swipe**: scorrere il dito sullo schermo per navigare tra pagine o contenuti.
- **Pinch**: avvicinare o allontanare due dita per ridurre o ingrandire un contenuto (zoom).

62.4 Osservazioni

- L'interazione touchscreen rende l'uso dei dispositivi mobili più intuitivo e naturale.
- Permette un controllo diretto degli elementi grafici senza dispositivi di input aggiuntivi.
- I sistemi operativi mobili integrano il supporto per numerosi gesti multi-touch per migliorare l'esperienza utente.

63 System Call

63.1 Introduzione

Le **system call** costituiscono l'interfaccia tra i programmi applicativi e i servizi forniti dal sistema operativo. Attraverso le system call, i programmi possono richiedere operazioni che coinvolgono direttamente l'hardware o le risorse di sistema.

- Generalmente sono implementate come routine scritte in **C o C++**.
- Alcune operazioni di basso livello possono richiedere l'uso del **linguaggio assembly**.
- Sono utilizzate per attività come l'accesso ai dispositivi hardware e la gestione delle risorse del sistema.

63.2 Categorie di System Call (System Call Categories)

Le **system call** possono essere classificate in diverse categorie in base al tipo di servizio del sistema operativo che forniscono. Generalmente vengono suddivise in sei categorie principali.

63.2.1 Process Control

- Creazione e terminazione di processi.
- Caricamento ed esecuzione di programmi.
- Attesa della terminazione di un processo.
- Allocazione e liberazione di memoria.

63.2.2 File Management

- Creazione e cancellazione di file.
- Apertura e chiusura di file.
- Lettura e scrittura di dati nei file.
- Ricerca e modifica delle proprietà dei file.

63.2.3 Device Management

- Richiesta e rilascio di dispositivi hardware.
- Lettura e scrittura su dispositivi di I/O.
- Impostazione dei parametri dei dispositivi.

63.2.4 Information Management

- Ottenere o impostare informazioni di sistema.
- Accesso a dati su processi, file e risorse.
- Gestione di attributi di sistema e statistiche.

63.2.5 Communication

- Creazione e gestione di connessioni tra processi.
- Scambio di dati tramite:
  - memoria condivisa
  - passaggio di messaggi
- Comunicazione tra processi nello stesso sistema o in rete.

63.2.6 Protection

- Controllo dell'accesso alle risorse di sistema.
- Gestione dei permessi per file, processi e dispositivi.
- Implementazione di meccanismi di sicurezza e autenticazione.

63.3 Osservazioni

- La classificazione delle system call aiuta a comprendere come il sistema operativo organizza i propri servizi.
- Ogni categoria rappresenta un insieme di operazioni fondamentali per la gestione delle risorse e il funzionamento del sistema.
- I programmi applicativi utilizzano queste system call indirettamente tramite API o librerie di sistema.

63.4 Esempio: Programma di Copia di File

Consideriamo il comando:

```
cp in.txt out.txt
```

Obiettivo: copiare i dati dal file `in.txt` al file `out.txt`.

63.4.1 Sequenza Tipica di System Call

Anche un'operazione semplice come la copia di un file richiede diverse system call:

- Visualizzare il prompt all'utente (scrittura sullo schermo).
- Leggere i nomi dei file di input e output (lettura da tastiera o selezione da interfaccia grafica).
- Aprire il file di input.
  - Se il file non esiste o l'accesso è negato → mostrare un messaggio di errore e interrompere l'operazione.
- Creare il file di output.
  - Se il file esiste già → decidere se sovrascriverlo o interrompere l'operazione.

63.4.2 Ciclo di Copia dei Dati

- Leggere dati dal file di input.
- Scrivere i dati nel file di output.
- Gestire eventuali errori di I/O.

63.4.3 Terminazione dell'Operazione

- Chiudere entrambi i file.
- Visualizzare un messaggio di successo.
- Terminare il programma.

63.4.4 Osservazione Chiave

- Anche un'operazione apparentemente semplice come la copia di un file richiede molte **system call**.
- Queste operazioni gestiscono:
  - Input/Output
  - Gestione degli errori
  - Accesso ai file
  - Terminazione del programma
- Le system call permettono ai programmi di utilizzare le funzionalità del sistema operativo in modo controllato e sicuro.

64 API e System Call (APIs and System Calls)

64.1 Perché i Programmatori Usano le API

Le **Application Programming Interfaces (API)** forniscono un livello di astrazione sopra le system call del sistema operativo. Invece di invocare direttamente le system call, i programmatori utilizzano le API offerte da librerie o ambienti di sviluppo.

- Le API nascondono la complessità delle system call di basso livello.
- Migliorano la **portabilità** dei programmi tra diversi sistemi operativi.
- Sono più **semplici e sicure** da utilizzare rispetto alle system call dirette.

64.2 API Comuni

Alcune API molto diffuse includono:

- **Windows API**: utilizzata per sviluppare applicazioni per sistemi operativi Microsoft Windows.
- **POSIX API**: standard utilizzato nei sistemi Unix e Unix-like.
- **Java API**: utilizzata dai programmi Java tramite la Java Virtual Machine (JVM).

64.3 Run-Time Environment (RTE)

Il **Run-Time Environment (RTE)** è l'ambiente che supporta l'esecuzione dei programmi.

- Fornisce strumenti necessari per eseguire i programmi, come:
  - Compilatori
  - Interpreti
  - Librerie di sistema
- Traduce le chiamate alle API in **system call** che il sistema operativo può eseguire.

64.4 Osservazioni

- Le API permettono agli sviluppatori di scrivere programmi indipendenti dall'hardware e dal sistema operativo specifico.
- Il Run-Time Environment agisce come intermediario tra il programma e il sistema operativo.
- Questo approccio migliora la portabilità, la sicurezza e la manutenibilità del software.

65 Controllo dei Processi (Process Control)

Le system call di **controllo dei processi** permettono al sistema operativo di gestire il ciclo di vita dei processi.

65.1 Operazioni Principali

- Creare nuovi processi.
- Avviare l'esecuzione dei processi.
- Sospendere temporaneamente un processo.
- Terminare un processo.

Queste operazioni consentono al sistema operativo di controllare come i programmi vengono eseguiti e coordinati nel sistema.

65.2 Terminazione dei Processi

Un processo può terminare in diversi modi:

- **Terminazione normale**: il programma completa correttamente la propria esecuzione tramite una funzione come `end()`.
- **Terminazione anomala**: il processo termina a causa di errori o eccezioni, ad esempio tramite `abort()`.

65.3 Gestione dei Crash

- Se un programma si arresta in modo imprevisto, il sistema operativo può:
  - mostrare un **messaggio di errore**
  - generare un **memory dump** (copia dello stato della memoria del processo)
- Il memory dump può essere analizzato per individuare la causa del problema.

65.4 Ritorno al Sistema

- Dopo la terminazione del processo, il controllo ritorna alla **shell** o al programma che ha avviato il processo.

65.5 Codice di Uscita (Exit Code)

- Il processo restituisce un **exit code** al sistema operativo:
  - **0**: esecuzione completata con successo.
  - **Valore diverso da 0**: indica un errore o un livello specifico di errore.

65.6 Osservazioni

- I codici di uscita permettono alla shell o ad altri programmi di determinare se un'operazione è stata completata correttamente.
- Le system call di process control sono fondamentali per il multitasking e la gestione efficiente dei programmi in esecuzione.

66 System Call per la Gestione dei File (File Management System Calls)

Le system call per la **gestione dei file** permettono ai programmi di interagire con il file system del sistema operativo per creare, utilizzare, modificare ed eliminare file.

66.1 Creazione ed Eliminazione dei File

- `create()` : crea un nuovo file nel file system.
- `delete()` : elimina un file esistente dal file system.

66.2 Accesso ai File

- `open()` : apre un file per permettere operazioni successive.
- `close()` : chiude il file dopo che le operazioni sono state completate.

66.3 Operazioni sui File

- `read()` : legge dati da un file.
- `write()` : scrive dati in un file.
- `reposition()` : sposta il puntatore di lettura/scrittura in una posizione specifica del file.

66.4 Gestione degli Attributi dei File

- `get_file_attributes()` : ottiene informazioni sugli attributi del file (dimensione, permessi, data di creazione, ecc.).
- `set_file_attributes()` : modifica gli attributi del file.

66.5 Idea Chiave

- Le system call del file system permettono ai programmi di gestire i file in modo **sicuro ed efficiente**.
- Forniscono un'interfaccia standard per la gestione dei dati memorizzati su dispositivi di archiviazione.

67 System Call per la Gestione dei Dispositivi (Device Management System Calls)

I processi necessitano di diverse risorse per poter essere eseguiti correttamente, tra cui memoria, accesso ai file, spazio su disco e dispositivi di input/output. Se una risorsa richiesta non è disponibile, il processo deve attendere finché non diventa libera.

67.1 Controllo delle Risorse (Resource Control)

- Le risorse del sistema vengono trattate come **dispositivi**, che possono essere:
  - **Fisici** (es. stampanti, dischi, tastiere)
  - **Virtuali** (risorse gestite dal sistema operativo)
- Per utilizzare una risorsa, un processo deve richiederla tramite specifiche system call:
  - `request()` : richiede l'accesso a un dispositivo.
  - `release()` : rilascia il dispositivo quando non è più necessario.

67.2 Operazioni di I/O

Una volta assegnato il dispositivo al processo, è possibile eseguire operazioni di input/output:

- `read()` : leggere dati dal dispositivo.
- `write()` : scrivere dati sul dispositivo.
- `reposition()` : cambiare la posizione di lettura o scrittura sul dispositivo.

67.3 Unificazione tra File e Dispositivi

Molti sistemi operativi, come **UNIX**, trattano file e dispositivi in modo simile. Questo significa che le stesse operazioni utilizzate per i file possono essere applicate anche ai dispositivi di I/O.

67.4 Idea Chiave

Il sistema operativo è responsabile dell'**allocazione e del controllo dei dispositivi**, garantendo che i processi possano accedere alle risorse in modo **sicuro, ordinato ed efficiente**.

68 System Call per la Gestione delle Informazioni (Information Management System Calls)

Le system call per la **gestione delle informazioni** permettono il trasferimento di dati tra i programmi e il sistema operativo, consentendo di ottenere informazioni di sistema, monitorare processi e supportare attività di debug.

68.1 Esempi di System Call di Informazioni Generali

- `time()` e `date()` : restituiscono l'ora e la data correnti.
- Informazioni di sistema:
  - Versione del sistema operativo
  - Utenti attualmente connessi
  - Memoria disponibile
  - Spazio su disco libero

68.2 Debug e Analisi

- **Memory dump** : cattura dello stato della memoria di un processo o del sistema.
- Tracciamento e esecuzione passo-passo tramite **traps**.
- Interruzioni da timer per registrare il tempo di CPU e abilitare il **profiling** delle prestazioni dei programmi.

68.3 Informazioni sui Processi

- Accesso e modifica degli attributi dei processi in esecuzione.
- Esempi di system call:
  - `get_process_attributes()` : legge attributi come priorità, stato, uso della memoria.
  - `set_process_attributes()` : modifica attributi dei processi secondo le necessità del sistema.

68.4 Idea Chiave

- Le system call di gestione delle informazioni consentono ai programmi di ottenere dati critici sul sistema, sui processi e sulle risorse.
- Supportano il monitoraggio delle prestazioni, il debug, la profilazione e la gestione sicura dei processi.

69 System Call per la Comunicazione (Communication System Calls)

Le system call di comunicazione permettono ai processi di scambiare informazioni tra loro, sia all'interno dello stesso sistema che tra sistemi diversi. Esistono due principali meccanismi: **Message Passing** e **Shared Memory**.

69.1 Message Passing

Il **message passing** consente lo scambio di messaggi tra processi:

- **Modalità diretta**: un processo invia un messaggio a un altro specificando l'identificativo del destinatario.
- **Modalità tramite mailbox**: i messaggi vengono scritti in una cassetta postale gestita dal sistema operativo, da cui i destinatari li leggono.

69.1.1 Identificazione

- Identificare host e processi:
  - `get_hostid()` : ottiene l'identificativo del computer.
  - `get_processid()` : ottiene l'identificativo del processo.

69.1.2 Gestione delle Connessioni

- `open()`, `close()` : aprire e chiudere risorse di comunicazione.
- `open_connection()`, `accept_connection()`, `close_connection()` : gestire le connessioni tra processi o sistemi remoti.

69.1.3 Comunicazione

- `read_message()`, `write_message()` : leggere e scrivere messaggi.
- I **daemon di comunicazione** possono usare `wait_for_connection()` per attendere richieste dai client.
- Ruoli:
  - **Client**: inizia la comunicazione.
  - **Server**: riceve e gestisce le richieste.

69.2 Shared Memory

Il **shared memory** permette a più processi di accedere a una regione di memoria condivisa:

- Creazione e accesso a regioni condivise tramite `shared_memory()`, `attach()`, `detach()`.
- Normalmente, la memoria dei processi è isolata; la shared memory richiede **sincronizzazione** per evitare conflitti.
- Molto veloce quando i processi si trovano sulla stessa macchina.

69.3 Idea Chiave

- Il message passing è flessibile e può funzionare sia tra processi locali che remoti.
- La memoria condivisa è estremamente efficiente per processi sullo stesso host, ma richiede meccanismi di sincronizzazione.
- Entrambi i metodi consentono al sistema operativo di garantire comunicazione sicura e coordinata tra processi.

70 System Call per la Protezione (Protection System Calls)

Le system call per la **protezione** consentono al sistema operativo di controllare l'accesso alle risorse, garantendo che solo utenti e processi autorizzati possano utilizzarle. Questi meccanismi sono fondamentali non solo nei sistemi multiutente, ma in qualsiasi ambiente operativo.

70.1 Operazioni Tipiche

- Modifica e interrogazione dei permessi:
  - `set_permission()` : imposta i permessi su file, dispositivi o altre risorse.
  - `get_permission()` : legge i permessi attuali di una risorsa.
- Controllo dell'accesso degli utenti:
  - `allow_user()` : concede a un utente o processo l'accesso a una risorsa.
  - `deny_user()` : nega l'accesso a una risorsa.

70.2 Obiettivo Principale

- Garantire che solo utenti e processi autorizzati possano accedere a file, dischi e altre risorse di sistema.
- Prevenire accessi non autorizzati che potrebbero compromettere l'integrità, la sicurezza o la stabilità del sistema.

70.3 Osservazioni

- Queste system call supportano la sicurezza a livello di kernel, fornendo un controllo centralizzato delle autorizzazioni.
- La gestione accurata dei permessi è essenziale anche nei sistemi con un solo utente per prevenire errori accidentali e proteggere risorse critiche.

71 Caricamento, Collegamento e Rilocazione (Loading, Linking, and Relocation)

Per eseguire un programma, il sistema operativo deve gestire diverse fasi preliminari che trasformano il codice sorgente in un processo eseguibile in memoria.

71.1 Fasi del Processo di Caricamento

1. **File oggetto relocabile:** I programmi iniziano come file oggetto compilati dal codice sorgente. Questi file contengono istruzioni e dati, ma gli indirizzi finali in memoria non sono ancora definiti.

2. **Collegamento (Linking):** Il linker combina più file oggetto e librerie in un unico eseguibile. Durante questa fase:

   - Vengono risolti riferimenti a funzioni e variabili esterne.
   - Si creano le tabelle dei simboli necessarie per riferimenti interni ed esterni.

3. **Rilocazione (Relocation):** La rilocazione assegna indirizzi finali di memoria alle istruzioni e ai dati. Eventuali riferimenti a indirizzi vengono corretti per riflettere le posizioni effettive in memoria.

4. **Caricamento (Loading):** Il loader legge l'eseguibile e lo carica nella memoria principale. Inoltre, prepara il **contesto del processo**, compreso il Program Counter, gli stack e le variabili di ambiente.

71.2 Esempio in UNIX

- `fork()` : crea un nuovo processo duplicando quello corrente.
- `exec()` : sostituisce lo spazio di indirizzi del processo con un nuovo eseguibile.
- **Dynamic linking** : le librerie vengono caricate solo se necessarie durante l'esecuzione, riducendo l'occupazione di memoria e i tempi di avvio.

71.3 Formati Eseguibili

- Gli eseguibili contengono codice macchina pronto all'esecuzione e tabelle dei simboli per la gestione di riferimenti e librerie.
- Esempio comune: **ELF (Executable and Linkable Format)**.

71.4 Idea Chiave

- Il caricamento, il collegamento e la rilocazione sono fondamentali per trasformare il codice sorgente in un processo eseguibile pronto a operare in memoria.
- La gestione dinamica delle librerie ottimizza risorse di memoria e tempi di esecuzione.

72 Struttura del Sistema Operativo: Design Monolitico (OS Structure: Monolithic Design)

Nei sistemi operativi di grandi dimensioni, è necessario progettare attentamente la struttura e modularizzare le funzioni. Tuttavia, il design monolitico rappresenta un approccio in cui tutte le funzionalità principali del kernel sono combinate in un unico spazio.

72.1 Concetto di Modulo

- **Modulo**: componente piccolo e autonomo del sistema, con interfacce e funzioni chiaramente definite.
- La modularizzazione aiuta nella manutenzione e nella comprensione del sistema, anche se nel design monolitico i moduli non sono separati a livello eseguibile.

72.2 Struttura Monolitica

- Tutte le funzionalità del kernel sono contenute in un unico binario statico.
- Un singolo **spazio di indirizzi** gestisce tutti i servizi core del sistema operativo.

72.3 Esempio: UNIX Originale

- Due parti principali:
  - **Kernel** : include driver dei dispositivi, interfaccia delle system call, gestione del file system, scheduling della CPU, gestione della memoria e altre funzionalità.
  - **Programmi di sistema** : utilities e strumenti che supportano l'esecuzione dei programmi utente.

72.4 Idea Chiave

- Nel design monolitico, numerose funzioni eterogenee del sistema operativo sono combinate in un unico **spazio kernel**.
- Questo approccio offre prestazioni elevate e accesso diretto a tutte le risorse, ma può rendere il sistema meno modulare e più difficile da mantenere.

73 Linux e Kernel Monolitici (Linux and Monolithic Kernels)

Linux si ispira ai principi di progettazione di UNIX, adottando un kernel monolitico che gestisce tutte le funzionalità principali in un unico spazio di indirizzi.

73.1 Comunicazione tra Applicazioni e Kernel

- Le applicazioni utente comunicano con il kernel tramite **glibc**, la libreria standard C, che fornisce interfacce alle system call.
- Questo approccio separa il codice utente dal kernel, pur permettendo l'accesso completo ai servizi di sistema.

73.2 Kernel Monolitico

- Funziona completamente in **modalità kernel** con un singolo spazio di indirizzi.
- Tutti i servizi core del sistema operativo (gestione processi, memoria, I/O, file system) risiedono nel kernel.
- Implementare e estendere un kernel monolitico può essere complesso a causa della concentrazione di tutte le funzionalità in un unico modulo.

73.3 Vantaggi del Kernel Monolitico

- **Prestazioni elevate** grazie all'esecuzione diretta di tutte le funzioni del kernel.
- **Basso overhead delle system call**, poiché non ci sono layer intermedi tra servizi kernel diversi.
- **Comunicazione interna rapida** tra componenti del kernel.

73.4 Esempi di Sistemi con Kernel Monolitico

- UNIX
- Linux
- Windows (alcune versioni adottano un approccio simile per motivi di velocità ed efficienza)

73.5 Idea Chiave

- Il kernel monolitico concentra tutte le funzionalità essenziali in un unico spazio, massimizzando le prestazioni e riducendo l'overhead delle chiamate di sistema, ma a costo di maggiore complessità nella manutenzione e nell'estendibilità.

74 Design a Strati e Modulare del Sistema Operativo (Layered and Modular OS Design)

Per migliorare la manutenibilità e la modularità dei sistemi operativi, si utilizzano approcci **layered** (a strati) e **modulari**, che isolano componenti e facilitano il debug.

74.1 Sistemi Tightly vs Loosely Coupled

- **Monolitico (tightly coupled)**: Tutte le componenti sono strettamente collegate; modifiche in una parte possono influenzare molte altre.
- **Modulare (loosely coupled)**: Le componenti sono isolate; le modifiche a un modulo hanno impatto limitato sul resto del sistema. Ciò rende più semplice la manutenzione e l'estensione.

74.2 Approccio a Strati (Layered Approach)

- Il sistema operativo è diviso in **livelli** (layers):
  - **Layer 0** : hardware.
  - **Layer N** : interfaccia utente.
- Ogni layer incapsula dati e operazioni, esponendo solo interfacce agli strati superiori.
- I layer superiori possono chiamare routine dei layer inferiori, ma non viceversa, garantendo isolamento e controllo.

74.3 Vantaggi del Design a Strati e Modulare

- Design più semplice e chiaro.
- Debug e verifica più facili: ogni layer può essere costruito e testato indipendentemente.
- Errori isolati al layer corrente, riducendo l'impatto sul sistema complessivo.

74.4 Idea Chiave

- Il design a strati e modulare migliora la manutenibilità, la chiarezza e l'affidabilità del sistema operativo rispetto a un kernel monolitico.
- Favorisce l'evoluzione e l'aggiornamento di componenti specifici senza compromettere l'intero sistema.

75 Architettura a Microkernel (Microkernel Architecture)

L'architettura a microkernel è stata sviluppata per superare i limiti dei kernel monolitici, come quello di UNIX, spostando gran parte dei servizi di sistema fuori dal kernel e riducendo le funzionalità core al minimo indispensabile.

75.1 Origine ed Esempi

- **Origine:** superare le limitazioni dei sistemi monolitici.
- **Esempio: Mach**, sviluppato presso la **Carnegie Mellon University**.

75.2 Caratteristiche Chiave

- **Kernel minimale:** gestisce solo i servizi essenziali come:
  - Gestione dei processi
  - Gestione della memoria
  - Comunicazione tra processi
- **Altri servizi** (file system, driver, gestione I/O avanzata) vengono eseguiti in **user space**.
- La comunicazione tra kernel e servizi utente avviene tramite **message passing**.

75.3 Vantaggi

- **Facile estendibilità:** nuovi servizi possono essere aggiunti nello user space senza modificare il kernel.
- **Portabilità migliorata:** cambiamenti al kernel minimo, quindi più semplice adattare il sistema a nuove piattaforme hardware.
- **Maggiore sicurezza e affidabilità:** un errore in un servizio utente non compromette l'intero sistema operativo.

75.4 Idea Chiave

- Ridurre il kernel al minimo e spostare la maggior parte dei servizi nello user space aumenta modularità, flessibilità e sicurezza.
- L'approccio microkernel facilita la manutenzione e l'evoluzione del sistema operativo rispetto ai kernel monolitici.

76 Moduli Caricabili del Kernel (Loadable Kernel Modules)

L'approccio dei **Loadable Kernel Modules** combina i vantaggi di un kernel monolitico e di un sistema modulare, permettendo di estendere le funzionalità del kernel senza ricompilarlo.

76.1 Concetto Chiave

- Il kernel contiene solo i **componenti core**, essenziali per il funzionamento del sistema.
- Servizi aggiuntivi possono essere caricati **dinamicamente** al momento del boot o durante l'esecuzione del sistema.
- Non è necessaria la ricompilazione del kernel per aggiungere nuove funzionalità.

76.2 Esempio di Moduli e Funzionalità

- **Kernel core:** gestione CPU, gestione memoria, pianificazione dei processi.
- **Moduli caricabili:** supporto per file system aggiuntivi, driver di dispositivi, protocolli di rete.

76.3 Vantaggi

- **Flessibile** come un microkernel, ma con struttura più efficiente.
- **Efficiente**: nessun overhead di message passing tipico dei microkernel.
- **Modulare e estensibile**: facilita la manutenzione e l'aggiornamento di componenti specifici senza interrompere il sistema.

76.4 Idea Chiave

- L'uso di moduli caricabili consente di combinare l'efficienza di un kernel monolitico con la flessibilità e la modularità dei sistemi a strati o microkernel, permettendo un sistema operativo più robusto e facilmente estendibile.

77 Sistemi Operativi Ibridi (Hybrid Operating Systems)

Molti sistemi operativi moderni combinano diversi modelli architetturali per ottenere un equilibrio ottimale tra prestazioni, sicurezza e usabilità.

77.1 Concetto Chiave

- Gli OS ibridi integrano caratteristiche di kernel monolitico, moduli caricabili e microkernel.
- L'obiettivo principale è ottenere il meglio dei vari approcci: alta efficienza, modularità e sicurezza.

77.2 Esempi di Sistemi Ibridi

- **Linux e Solaris**
  - Kernel monolitico per massimizzare le prestazioni.
  - Supporto a moduli dinamici per estendere funzionalità senza ricompilare il kernel.
- **Windows**
  - Principalmente monolitico.
  - Alcune funzionalità seguono lo stile microkernel, con sottosistemi che operano in modalità utente.
  - Supporta moduli caricabili dinamicamente per estensioni e driver.

77.3 Idea Chiave

- I moderni sistemi operativi sono ibridi per combinare prestazioni elevate, modularità e sicurezza, integrando concetti di kernel monolitico, modulare e microkernel in un'unica architettura coerente.

78 Generazione e Configurazione del Sistema Operativo (OS Generation & Configuration)

La costruzione e la configurazione di un sistema operativo richiedono una serie di passaggi accurati per adattare l'OS all'hardware e alle esigenze dell'utente.

78.1 Fasi per Costruire un Sistema Operativo

1. Scrivere il **codice sorgente**.
2. Configurare l'OS per il sistema target.
3. Compilare il kernel e i moduli.
4. Installare il sistema operativo sul dispositivo.
5. Avviare (**boot**) e far funzionare l'OS.

78.2 Metodi di Configurazione

- **Ricompilazione completa (Full rebuild)**
  - Il file di configurazione modifica il codice sorgente.
  - Il kernel viene ricompilato per adattarsi esattamente all'hardware.
- **Moduli precompilati (Precompiled modules)**
  - Selezione e collegamento solo dei componenti necessari, come i driver.
  - Più veloce, ma meno specializzato rispetto a una ricompilazione completa.
- **Sistemi completamente modulari (Fully modular systems)**
  - I moduli vengono scelti e caricati al momento dell'esecuzione (**runtime**).
  - Offre la massima flessibilità e adattabilità all'ambiente hardware.

78.3 Idea Chiave

- La configurazione di un sistema operativo può avvenire sia **a tempo di compilazione** che **a tempo di esecuzione**, bilanciando flessibilità, velocità e supporto all'hardware specifico.